

ML модель предсказания возрастной группы

Цели и задачи проекта

Цель:

Создать возможность автоматического определения возрастной группы пользователя для показа подходящей рекламы и снижения риска показа рекламы для взрослых несовершеннолетним.

Задача:

Построить модель классификации, которая по портрету пользователя предсказывает его возрастную категорию (до 18, 18-25, 26-40, 41-55, 56+).

План работы

- Этап 1 Подготовка среды: загрузка данных, фиксация random_state, установка библиотек.
- Этап 2 Анализ данных (EDA): изучаем признаки, ищем пропуски, дубликаты, выбросы, проверяем корреляции между признаками.
- Этап 3 Предобработка: заполняем пропуски, кодируем категории, масштабируем числа, собираем пайплайн. Делим данные 80/20 тестовую выборку не трогаем до самого конца.
- Этап 4 Отбор признаков: используем методы-обёртки для поиска оптимального набора признаков.
- Этап 5 Базовая модель: обучаем DummyClassifier и многоклассовые модели OvR, OvO, multinomial LogisticRegression, SVC модели.
- Этап 6 Новые признаки: генерируем дополнительные признаки на основе уже имеющихся, проверяем помогают ли они модели.
- Этап 7 Подбор гиперпараметров: перебираем параметры моделей, сравниваем результаты в таблице, выбираем лучшую конфигурацию.
- Этап 8 Финальная модель: обучаем лучшую модель на всех тренировочных данных, замеряем качество на тестовой выборке, сохраняем модель и пайплайн через joblib.
- Этап 9 Отчёт: описываем итоговые метрики и какие признаки сильнее всего влияют на возрастную группу.

Этап 1 Подготовка среды: загрузка данных, фиксация random_state, установка библиотек.

- Установим и настроим библиотеки
- Зафиксируем random_state и параметры отображения
- Загрузим данные из CSV-файлов:
 - ds_s13_users.csv
 - ds_s13_visits.csv
 - ads_activity
 - surf_depth
 - primary_device
 - cloud_usage

Установим и настроим библиотеки

```
In [1]: !pip install phik
        !pip install category_encoders
```

```
import numpy as np
import pandas as pd
import phik
import joblib
```

```
import matplotlib.pyplot as plt
import seaborn as sns
```

```
from sklearn.model_selection import (train_test_split, GridSearchCV, StratifiedKFold, cross_validate)
```

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
```

```

from sklearn.preprocessing import (OneHotEncoder, StandardScaler)
from sklearn.preprocessing import (MinMaxScaler, OrdinalEncoder)

from category_encoders import TargetEncoder
from sklearn.impute import SimpleImputer

from sklearn.feature_selection import (SelectKBest, mutual_info_classif, RFE)

from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC, SVC
from sklearn.dummy import DummyClassifier

from sklearn.metrics import (precision_score, recall_score, f1_score, classification_report, make_scorer, accuracy_score, confusion_matrix)

from sklearn.multiclass import (OneVsRestClassifier, OneVsOneClassifier)
from sklearn.model_selection import cross_val_predict
from sklearn.metrics import precision_recall_curve

```

Out:

```

Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: phik in C:\Users\user\AppData\Roaming\Python\Python313\site-packages (0.12.5)
Requirement already satisfied: numpy>=1.18.0 in C:\Users\user\AppData\Roaming\Python\Python313\site-packages (from phik) (2.4.6)
Requirement already satisfied: scipy>=1.5.2 in C:\Users\user\AppData\Roaming\Python\Python313\site-packages (from phik) (1.18.0)
Requirement already satisfied: pandas>=0.25.1 in C:\Users\user\AppData\Roaming\Python\Python313\site-packages (from phik) (3.0.1)
Requirement already satisfied: matplotlib>=2.2.3 in C:\Users\user\AppData\Roaming\Python\Python313\site-packages (from phik) (3.10.8)
Requirement already satisfied: joblib>=0.14.1 in C:\Users\user\AppData\Roaming\Python\Python313\site-packages (from phik) (1.5.3)
Requirement already satisfied: contourpy>=1.0.1 in C:\Users\user\AppData\Roaming\Python\Python313\site-packages (from matplotlib>=2.2.3->phik) (1.3.3)
Requirement already satisfied: cycler>=0.10 in C:\Users\user\AppData\Roaming\Python\Python313\site-packages (from matplotlib>=2.2.3->phik) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in C:\Users\user\AppData\Roaming\Python\Python313\site-packages (from matplotlib>=2.2.3->phik) (4.61.1)
Requirement already satisfied: kiwisolver>=1.3.1 in C:\Users\user\AppData\Roaming\Python\Python313\site-packages (from matplotlib>=2.2.3->phik) (1.4.9)
Requirement already satisfied: packaging>=20.0 in C:\Users\user\AppData\Roaming\Python\Python313\site-packages (from matplotlib>=2.2.3->phik) (26.0)
Requirement already satisfied: pillow>=8 in C:\Users\user\AppData\Roaming\Python\Python313\site-packages (from matplotlib>=2.2.3->phik) (12.1.1)
Requirement already satisfied: pyparsing>=3 in C:\Users\user\AppData\Roaming\Python\Python313\site-packages (from matplotlib>=2.2.3->phik) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in C:\Users\user\AppData\Roaming\Python\Python313\site-packages (from matplotlib>=2.2.3->phik) (2.9.0.post0)
Requirement already satisfied: tzdata in C:\Users\user\AppData\Roaming\Python\Python313\site-packages (from pandas>=0.25.1->phik) (2025.3)
Requirement already satisfied: six>=1.5 in C:\Users\user\AppData\Roaming\Python\Python313\site-packages (from python-dateutil>=2.7->matplotlib>=2.2.3->phik) (1.17.0)

```

```

[notice] A new release of pip is available: 26.0.1 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip

```

```

Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: category_encoders in C:\Users\user\AppData\Roaming\Python\Python313\site-packages (2.9.0)
Requirement already satisfied: numpy>=1.14.0 in C:\Users\user\AppData\Roaming\Python\Python313\site-packages (from category_encoders) (2.4.6)
Requirement already satisfied: pandas>=1.0.5 in C:\Users\user\AppData\Roaming\Python\Python313\site-packages (from category_encoders) (3.0.1)
Requirement already satisfied: patsy>=0.5.1 in C:\Users\user\AppData\Roaming\Python\Python313\site-packages (from category_encoders) (1.0.2)
Requirement already satisfied: scikit-learn>=1.6.0 in C:\Users\user\AppData\Roaming\Python\Python313\site-packages (from category_encoders) (1.9.0)
Requirement already satisfied: scipy>=1.0.0 in C:\Users\user\AppData\Roaming\Python\Python313\site-packages (from category_encoders) (1.18.0)
Requirement already satisfied: statsmodels>=0.9.0 in C:\Users\user\AppData\Roaming\Python\Python313\site-packages (from category_encoders) (0.14.6)
Requirement already satisfied: python-dateutil>=2.8.2 in C:\Users\user\AppData\Roaming\Python\Python313\site-packages (from pandas>=1.0.5->category_encoders) (2.9.0.post0)
Requirement already satisfied: tzdata in C:\Users\user\AppData\Roaming\Python\Python313\site-packages (from pandas>=1.0.5->category_encoders) (2025.3)

```

```
Requirement already satisfied: six>=1.5 in C:\Users\user\AppData\Roaming\Python\Python313\site-packages (from python-dateutil>=2.8.2->pandas
>=1.0.5->category_encoders) (1.17.0)
Requirement already satisfied: joblib>=1.4.0 in C:\Users\user\AppData\Roaming\Python\Python313\site-packages (from scikit-learn>=1.6.0->cate
gory_encoders) (1.5.3)
Requirement already satisfied: narwhals>=2.0.1 in C:\Users\user\AppData\Roaming\Python\Python313\site-packages (from scikit-learn>=1.6.0->ca
teory_encoders) (2.23.0)
Requirement already satisfied: threadpoolctl>=3.5.0 in C:\Users\user\AppData\Roaming\Python\Python313\site-packages (from scikit-learn>=1.6.
0->category_encoders) (3.6.0)
Requirement already satisfied: packaging>=21.3 in C:\Users\user\AppData\Roaming\Python\Python313\site-packages (from statsmodels>=0.9.0->cat
egory_encoders) (26.0)
```

```
[notice] A new release of pip is available: 26.0.1 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

Используемые библиотеки

- numpy - работа с многомерными массивами и математическими операциями
- pandas - работа с таблицами и анализ данных
- phik - анализ нелинейных зависимостей между признаками с помощью коэффициента корреляции Phik

Визуализация данных

- matplotlib - построение графиков и визуализация данных
- seaborn - визуализация распределений, корреляций и категориальных признаков

Разделение данных и валидация

- train_test_split - разделение данных на обучающую и тестовую выборки
- GridSearchCV - подбор гиперпараметров моделей
- StratifiedKFold - кросс-валидация
- cross_validate - оценка модели по нескольким метрикам

Предобработка данных и пайплайны

- ColumnTransformer - применение преобразований к группам признаков
- Pipeline - объединение этапов предобработки и модели в единую цепочку
- OneHotEncoder - кодирование категориальных признаков
- StandardScaler - стандартизация числовых признаков
- TargetEncoder - кодирование категориальных признаков
- SimpleImputer - заполнение пропусков

Отбор признаков

- VarianceThreshold - удаление признаков с низкой дисперсией
- SelectKBest - выбор наиболее информативных признаков
- mutual_info_classif - оценка связи между признаками и целевой переменной
- RFE - рекурсивный отбор признаков
- SequentialFeatureSelector - последовательный отбор признаков

Модели машинного обучения

- LogisticRegression - модель логистической регрессии для классификации
- LinearSVC - линейная модель опорных векторов
- SVC - модель опорных векторов с разными ядрами
- DummyClassifier - базовая модель для сравнения качества

Метрики качества

- average_precision_score - точность модели(сколько попали)

- `precision_score` - сколько из предсказанных относится к классу
- `recall_score` - сколько объектов класса выявили из общего количества
- `f1_score` - баланс `precision` и `recall`
- `classification_report` - подробный отчёт по качеству

Зафиксируем `random_state` и параметры отображения

```
In [2]: plt.style.use('default')
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['font.size'] = 12

RANDOM_STATE = 42
```

- `plt.style.use('default')` - использование стандартного стиля визуализации `matplotlib`.
- `plt.rcParams['figure.figsize'] = (10, 6)` - размер графиков.
- `plt.rcParams['font.size'] = 12` - размер шрифта на графиках.
- `RANDOM_STATE = 42` - фиксация генератора случайных чисел для воспроизводимости результатов.

Загрузим данные из CSV-файлов:

- `ds_s13_users.csv`
- `ds_s13_visits.csv`
- `ads_activity`
- `surf_depth`
- `primary_device`
- `cloud_usage`

```
In [3]: try:
    users = pd.read_csv('ds_s13_users.csv')
    visits = pd.read_csv('ds_s13_visits.csv')
    ads_activity = pd.read_csv('ads_activity.csv')
    surf_depth = pd.read_csv('surf_depth.csv')
    primary_device = pd.read_csv('primary_device.csv')
    cloud_usage = pd.read_csv('cloud_usage.csv')

except:
    users = pd.read_csv('https://code.s3.yandex.net/datasets/ds_s13_users.csv')
    visits = pd.read_csv('https://code.s3.yandex.net/datasets/ds_s13_visits.csv')
    ads_activity = pd.read_csv('https://code.s3.yandex.net/datasets/ads_activity.csv')
    surf_depth = pd.read_csv('https://code.s3.yandex.net/datasets/surf_depth.csv')
    primary_device = pd.read_csv('https://code.s3.yandex.net/datasets/primary_device.csv')
    cloud_usage = pd.read_csv('https://code.s3.yandex.net/datasets/cloud_usage.csv')
```

Проверка выгруженных данных

```
In [4]: datasets = {
    'users': users,
    'visits': visits,
    'ads_activity': ads_activity,
    'surf_depth': surf_depth,
    'primary_device': primary_device,
    'cloud_usage': cloud_usage
}
for name, df in datasets.items():
    print(name, df.shape)
```

```
Out:
users (5913, 2)
visits (1065745, 5)
ads_activity (5826, 2)
```

```
surf_depth (5715, 2)
primary_device (5669, 2)
cloud_usage (5680, 2)
```

данные выгружены корректно, можно переходить к EDA

Этап 2 Анализ данных (EDA): изучаем признаки, ищем пропуски, дубликаты, выбросы, проверяем корреляции между признаками.

Первичный анализ выгруженных данных

```
In [5]: def datasets_info(datasets):
        for name, df in datasets.items():
            print(name)

            for col in df.columns:
                print(f"\t{col}"
                      f"\n\t\tтип: {df[col].dtype}"
                      f"\n\t\tвсего строк: {len(df[col])}"
                      f"\n\t\tпропусков: {df[col].isna().sum()}"
                      f"\n\t\tпримеры возможных значений: {list(df[col].dropna().unique()[:5])}")

            print()
        datasets_info(datasets)

Out:
users
    user_id
        тип: str
        всего строк: 5913
        пропусков: 0
        примеры возможных значений: ['f545-8c95aefe8d3e5548a689-a5b2fd39', 'cb48-5a0d6cde4d86ae10637e-c8ceb6ed', '678b-614cd47d854b9
d591db2-000b2e50', '4ac0-dad169100b4a29b20818-b26ae7c5', 'f19b-9ac21ca973b41ecfa8c3-6a58191d']
    age_category
        тип: int64
        всего строк: 5913
        пропусков: 0
        примеры возможных значений: [np.int64(4), np.int64(2), np.int64(0), np.int64(1), np.int64(3)]

visits
    date
        тип: str
        всего строк: 1065745
        пропусков: 0
        примеры возможных значений: ['2025-11-01', '2025-11-02', '2025-11-03', '2025-11-04', '2025-11-05']
    daytime
        тип: str
        всего строк: 1065745
        пропусков: 0
        примеры возможных значений: ['вечер', 'день', 'ночь', 'утро']
    session_id
        тип: str
        всего строк: 1065745
        пропусков: 0
        примеры возможных значений: ['066e4e02-8c1f-45eb-a50f-178659abe698', '0bce1749-3376-439c-9a22-f8ffbbba00e9a', '3445d8c4-221d-
4d88-bb6a-a2939fe3c610', '3bf97286-1d91-4aaa-af4a-ed58eceb8cd2', '40e22712-3cad-410d-a9f0-13bd8f6911c0']
    user_id
        тип: str
        всего строк: 1065745
        пропусков: 0
        примеры возможных значений: ['0010-5cf8f6b38a7b6c70a021-009dbcd4', '0013-4ae5f7d127b91a3fb0f8-ba59f141', '0014-d3032d60979a8
d2b3077-f09bdce8', '001a-eee53e44f848608779b0-78704a67', '002c-40a064b12e1217e12207-a56eaf3b']
    website_category
        тип: str
        всего строк: 1065745
        пропусков: 0
        примеры возможных значений: ['Category 17', 'Category 19', 'Category 18', 'Category 20', 'Category 05']

ads_activity
```

```

user_id
    тип: str
    всего строк: 5826
    пропусков: 0
    примеры возможных значений: ['e318-d8e69c86b543a5fb927c-c36fb6e6', '35cd-a972339dec534f49332c-a8b6d383', 'f7e6-3b29cf9cb7ed4
bb00d8f-81534360', '5186-e25a37549e50f45b2b43-178eaabe', 'febd-077f277466253ee04ef6-42656680']

ads_activity
    тип: str
    всего строк: 5826
    пропусков: 0
    примеры возможных значений: ['очень часто', 'редко', 'очень редко', 'умеренно', 'часто']

surf_depth
    user_id
        тип: str
        всего строк: 5715
        пропусков: 0
        примеры возможных значений: ['f238-0c4c1e787cce311541b7-736925a0', '9030-1b562ad80182b6dc27f1-ce811740', '22e0-7c6cadcc45e24
6b8688d-c43c9b23', '9d7f-a19f10756378940a49b5-5d03e1ef', '4233-bb5ae4b09827e5497094-1a4956af']
    surf_depth
        тип: str
        всего строк: 5715
        пропусков: 0
        примеры возможных значений: ['поверхностно', 'глубоко', 'средне']

primary_device
    user_id
        тип: str
        всего строк: 5669
        пропусков: 0
        примеры возможных значений: ['d602-ec060db7597a6b8cd4e7-aa625896', '9204-9558455be649d4e77945-b5e25d62', '5eea-22babd6a9474b
43b9d0b-a39a4cf2', 'c142-0296948e8d08e417de10-2da9523c', 'abec-bb4092da51eb2233a928-e44ba074']
    primary_device
        тип: str
        всего строк: 5669
        пропусков: 0
        примеры возможных значений: ['смартфон', 'ПК', 'ноутбук', 'планшет']

cloud_usage
    user_id
        тип: str
        всего строк: 5680
        пропусков: 0
        примеры возможных значений: ['a1e4-91c8a52eb855595e653f-298ce305', 'db9a-7b8e9e94448b7fcb19b6-4edca15f', '0d55-9ad768879e9b0
8ca7ff9-843f76c7', '4baa-43285d10a6d3cc969f2a-b21881d1', 'b8cd-cbb2411db005115ca64d-32700c62']
    cloud_usage
        тип: bool
        всего строк: 5680
        пропусков: 0
        примеры возможных значений: [np.False_, np.True_]

```

Первичный анализ по сырым данным

- 1. `user_id` присутствует во всех таблицах, однако не обо всех пользователях есть данные в источниках
 - `users` - 5913 уникальных `user_id`, является эталонной таблицей, так как содержит целевую переменную `age_category`
 - `ads_activity` - 5826 объектов, содержит важный признак CTR активности пользователя. При объединении с `users` теряется 87 пользователей, пропуски можно будет обработать отдельной категорией
 - `surf_depth` - 5715 объектов, показывает вовлеченность пользователей. При `merge` с `users` теряется 198 пользователей
 - `primary_device` - 5669 объектов, основной тип устройства пользователя. При объединении теряется 244 пользователя, может быть важным для определения пользователей старших групп заходящих с ПК
 - `cloud_usage` - 5680 объектов, показывает использование облачных сервисов пользователем. При `merge` с `users` теряется 233 пользователя
- 2. `visits` - 1065745 строк активность пользователей, самая большая таблица в датасете требуются агрегация до уровня `user_id` перед объединением с другими таблицами
 - Агрегирование по `user_id` - позволит перейти от уровня `session_id` к профилю пользователя
 - Доля посещений по `website_category` - будет рассчитана как частота посещения каждой категории сайтов для пользователя

- Распределение активности по `daytime` - выявим предпочтительное время пользователя (утро/день/вечер/ночь)
 - Количество сессий (`session_id.count()`) - активность пользователя
- 3. Типы данных
 - Категориальные признаки: `ads_activity` , `surf_depth` , `primary_device` , `daytime` , `website_category`
 - Числовые признаки: `age_category`
 - Бинарные признаки: `cloud_usage`
 - Временные признаки: `date` (требуется преобразование из `str` в `datetime`)
 - Идентификаторы: `user_id` , `session_id`

Создание единого датафрейма

Сгруппируем данные таблицы visits по user_id

- найдем долю каждого значения каждой категории по всем сессиям пользователя
- добавим активность пользователя по количеству сессий
- выявим сезонную активность пользователя

Доля каждого значения каждой категории по всем сессиям пользователя

```
In [6]: def create_profiles(df, group_col, cat_cols, num_cols):
        profiles = pd.DataFrame(index=df[group_col].unique())
        if num_cols:
            profiles = (df.groupby(group_col)[num_cols].agg(['mean', 'median']))
            profiles.columns = [f'{col}_{value}' for col, value in profiles.columns]

        for col in cat_cols:
            pivot = (pd.crosstab(df[group_col], df[col], normalize='index'))
            pivot.columns = [f'{col}_{value}' for value in pivot.columns]

            profiles = profiles.join(pivot)

        return profiles

profiles = create_profiles(df=visits, group_col='user_id', cat_cols=['daytime', 'website_category'], num_cols=[])
profiles.head(3)
```

Out:

	daytime_вечер	daytime_день	daytime_ночь	daytime_утро	website_category_Category 01	website_category_Category 02
0010-5cf8f6b38a7b6c70a021-009dbcda	0.409009	0.327928	0.097297	0.165766	0.028829	0.000000
0013-4ae5f7d127b91a3fb0f8-ba59f141	0.278075	0.368984	0.090909	0.262032	0.080214	0.058824
0014-d3032d60979a8d2b3077-f09bdce8	0.300000	0.391667	0.091667	0.216667	0.008333	0.016667

3 rows × 24 columns

Добавим активность по количеству сессий

```
In [7]: profiles = profiles.join(visits.groupby('user_id')['session_id'].count().rename('session_count'))
profiles.head(3)
```

Out:

	daytime_вечер	daytime_день	daytime_ночь	daytime_утро	website_category_Category 01	website_category_Category 01
0010-5cf8f6b38a7b6c70a021-009dbcda	0.409009	0.327928	0.097297	0.165766	0.028829	0.000000
0013-4ae5f7d127b91a3fb0f8-ba59f141	0.278075	0.368984	0.090909	0.262032	0.080214	0.058824
0014-d3032d60979a8d2b3077-f09bdce8	0.300000	0.391667	0.091667	0.216667	0.008333	0.016667

3 rows × 25 columns

Используем колонку date для выявления сезонности

рассмотрим какие периоды нам доступны предварительно преобразовав дату во временной формат

```
In [8]: visits['date'] = pd.to_datetime(visits['date'])
min_date = visits['date'].min()
max_date = visits['date'].max()

print("Минимальная дата:", min_date)
print("Максимальная дата:", max_date)
```

Out:

Минимальная дата: 2025-11-01 00:00:00
Максимальная дата: 2025-11-14 00:00:00

Временной период небольшой только 14 дней.

Тогда можно выцепить лишь изменения по активности в выходные дни и будние в дальнейшем добавим новый признак is_weekend пока просто присоединим каждый из 14 дней

```
In [9]: profiles = profiles.join(create_profiles(df=visits, group_col='user_id', cat_cols=['date'], num_cols=[]))
```

```
In [10]: profiles.head()
```

Out:

	daytime_вечер	daytime_день	daytime_ночь	daytime_утро	website_category_Category 01	website_category_Category 01
0010-5cf8f6b38a7b6c70a021-009dbcda	0.409009	0.327928	0.097297	0.165766	0.028829	0.000000
0013-4ae5f7d127b91a3fb0f8-ba59f141	0.278075	0.368984	0.090909	0.262032	0.080214	0.058824
0014-d3032d60979a8d2b3077-f09bdce8	0.300000	0.391667	0.091667	0.216667	0.008333	0.016667
001a-eee53e44f848608779b0-78704a67	0.415808	0.329897	0.109966	0.144330	0.065292	0.044674
002c-40a064b12e1217e12207-a56eaf3b	0.300000	0.384000	0.068000	0.248000	0.000000	0.056000

5 rows × 39 columns

Объединяем все в единый датасет

Перед объединением проверим данные на наличие дубликатов user_id

```
In [11]: print(ads_activity.duplicated().sum(), surf_depth.duplicated().sum(), primary_device.duplicated().sum(), cloud_usage.duplicated().sum(), profiles.duplicated().sum(), users.duplicated().sum())
```

Out:

```
233 0 0 0 0 87
```

найлены дубликаты в ads_activity и users их необходимо удалить

```
In [12]: users = users.drop_duplicates()
ads_activity = ads_activity.drop_duplicates()
```

```
In [13]: df = users.copy()
df = df.join(profiles, on='user_id')
df = df.merge(ads_activity, on='user_id', how='left')
df = df.merge(surf_depth, on='user_id', how='left')
df = df.merge(primary_device, on='user_id', how='left')
df = df.merge(cloud_usage, on='user_id', how='left')
```

```
In [14]: print(df.info())
```

Out:

```
<class 'pandas.DataFrame'>
RangeIndex: 5826 entries, 0 to 5825
Data columns (total 45 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   user_id                                   5826 non-null   str
1   age_category                             5826 non-null   int64
2   daytime_вечер                           5826 non-null   float64
3   daytime_день                             5826 non-null   float64
4   daytime_ночь                             5826 non-null   float64
5   daytime_утро                             5826 non-null   float64
6   website_category_Category 01             5826 non-null   float64
7   website_category_Category 02             5826 non-null   float64
8   website_category_Category 03             5826 non-null   float64
9   website_category_Category 04             5826 non-null   float64
10  website_category_Category 05             5826 non-null   float64
11  website_category_Category 06             5826 non-null   float64
12  website_category_Category 07             5826 non-null   float64
13  website_category_Category 08             5826 non-null   float64
14  website_category_Category 09             5826 non-null   float64
15  website_category_Category 10             5826 non-null   float64
16  website_category_Category 11             5826 non-null   float64
17  website_category_Category 12             5826 non-null   float64
18  website_category_Category 13             5826 non-null   float64
19  website_category_Category 14             5826 non-null   float64
20  website_category_Category 15             5826 non-null   float64
21  website_category_Category 16             5826 non-null   float64
22  website_category_Category 17             5826 non-null   float64
23  website_category_Category 18             5826 non-null   float64
24  website_category_Category 19             5826 non-null   float64
25  website_category_Category 20             5826 non-null   float64
26  session_count                           5826 non-null   int64
27  date_2025-11-01 00:00:00                 5826 non-null   float64
28  date_2025-11-02 00:00:00                 5826 non-null   float64
29  date_2025-11-03 00:00:00                 5826 non-null   float64
```

```

30 date_2025-11-04 00:00:00      5826 non-null   float64
31 date_2025-11-05 00:00:00      5826 non-null   float64
32 date_2025-11-06 00:00:00      5826 non-null   float64
33 date_2025-11-07 00:00:00      5826 non-null   float64
34 date_2025-11-08 00:00:00      5826 non-null   float64
35 date_2025-11-09 00:00:00      5826 non-null   float64
36 date_2025-11-10 00:00:00      5826 non-null   float64
37 date_2025-11-11 00:00:00      5826 non-null   float64
38 date_2025-11-12 00:00:00      5826 non-null   float64
39 date_2025-11-13 00:00:00      5826 non-null   float64
40 date_2025-11-14 00:00:00      5826 non-null   float64
41 ads_activity                  5593 non-null   str
42 surf_depth                    5715 non-null   str
43 primary_device                5669 non-null   str
44 cloud_usage                   5680 non-null   object
dtypes: float64(38), int64(2), object(1), str(4)
memory usage: 2.0+ MB
None

```

Итог формирование единого датасета

- 1. Исходные данные
 - users - базовая таблица, содержащая 5913 пользователей и целевую переменную age_category
 - visits - активность пользователей (более миллиона записей), требующая агрегации до уровня user_id
 - ads_activity, surf_depth, primary_device, cloud_usage - дополнительные поведенческие признаки пользователей
- 2. Что было сделано при объединении
 - Таблица visits была агрегирована по (user_id) засчет создания профиля:
 - доли посещений по каждому daytime
 - доли посещений по каждому website_category
 - распределение активности по дням
 - общее количество сессий (активность)
- 3. Итоговый датасет
 - Получен датафрейм размером 5826 пользователей
 - Каждая строка - уникальный user_id
- 4. Особенности объединения
 - Агрегация признаков выполнялась по user_id признаки, сформированные из visits, рассчитаны без использования информации из целевой переменной, так как разделение на train/test будет выполняться по user_id, исключается риск утечки данных
 - Были обнаружены и удалены дубликаты в users и ads_activity

Распределение целевой переменной

```

In [15]: def target_distribution(x):
          value_counts = x.value_counts()
          value_share = x.value_counts(normalize=True)
          print('Количество объектов:')
          print(value_counts)
          print('Доли классов:')
          print(value_share)

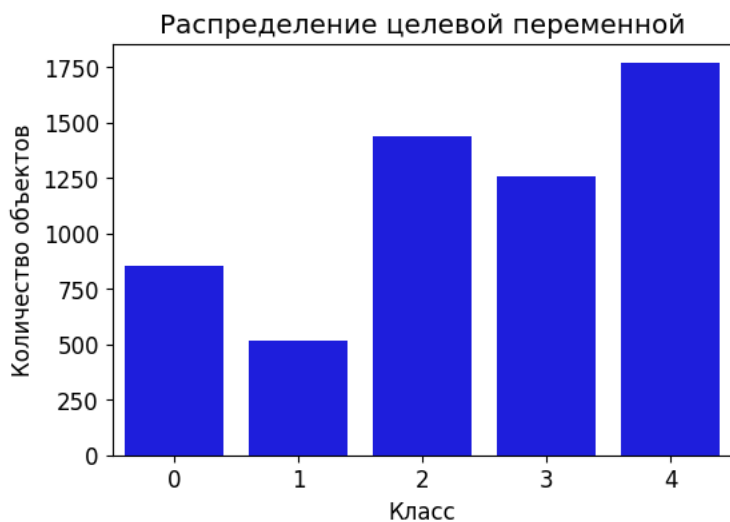
          plt.figure(figsize=(6, 4))
          sns.countplot(x=x, color='blue')
          plt.title('Распределение целевой переменной')
          plt.xlabel('Класс')
          plt.ylabel('Количество объектов')
          plt.show()

          target_distribution(df['age_category'])

```

Out:

```
Количество объектов:
age_category
4    1766
2    1439
3    1254
0     853
1     514
Name: count, dtype: int64
Доли классов:
age_category
4    0.303124
2    0.246996
3    0.215242
0    0.146413
1    0.088225
Name: proportion, dtype: float64
```



Анализ целевой переменной

- 1. Распределение классов
 - Класс 4 - 1766 объектов (30.31%)
 - Класс 2 - 1439 объектов (24.70%)
 - Класс 3 - 1254 объекта (21.52%)
 - Класс 0 - 853 объекта (14.64%)
 - Класс 1 - 514 объектов (8.82%)

```
<li>2. Вывод
  <ul>
    <li>Есть дисбаланс классов</li>
    <li>Меньше всего представлен класс 1</li>
    <li>Используем F1-масро</li>
    <li>Учитываем дисбаланс классов при обучении</li>
    <li>Используем stratified split</li>
  </ul>
</li>
```

Обработка пропусков

```
In [16]: def missing_report(df):
          total = len(df)
          for col in df.columns:
              print(col)
              print("\t", "пропусков:", df[col].isna().sum())
```

```
print("\t", f"доля: {round(df[col].isna().sum() / total * 100, 2)}%")
missing_report(df)
```

Out:

```
user_id
    пропусков: 0
    доля: 0.0%
age_category
    пропусков: 0
    доля: 0.0%
daytime_вечер
    пропусков: 0
    доля: 0.0%
daytime_день
    пропусков: 0
    доля: 0.0%
daytime_ночь
    пропусков: 0
    доля: 0.0%
daytime_утро
    пропусков: 0
    доля: 0.0%
website_category_Category 01
    пропусков: 0
    доля: 0.0%
website_category_Category 02
    пропусков: 0
    доля: 0.0%
website_category_Category 03
    пропусков: 0
    доля: 0.0%
website_category_Category 04
    пропусков: 0
    доля: 0.0%
website_category_Category 05
    пропусков: 0
    доля: 0.0%
website_category_Category 06
    пропусков: 0
    доля: 0.0%
website_category_Category 07
    пропусков: 0
    доля: 0.0%
website_category_Category 08
    пропусков: 0
    доля: 0.0%
website_category_Category 09
    пропусков: 0
    доля: 0.0%
website_category_Category 10
    пропусков: 0
    доля: 0.0%
website_category_Category 11
    пропусков: 0
    доля: 0.0%
website_category_Category 12
    пропусков: 0
    доля: 0.0%
website_category_Category 13
    пропусков: 0
    доля: 0.0%
website_category_Category 14
    пропусков: 0
    доля: 0.0%
website_category_Category 15
    пропусков: 0
    доля: 0.0%
website_category_Category 16
    пропусков: 0
    доля: 0.0%
website_category_Category 17
    пропусков: 0
    доля: 0.0%
```

```
website_category_Category 18
    пропусков: 0
    доля: 0.0%
website_category_Category 19
    пропусков: 0
    доля: 0.0%
website_category_Category 20
    пропусков: 0
    доля: 0.0%
session_count
    пропусков: 0
    доля: 0.0%
date_2025-11-01 00:00:00
    пропусков: 0
    доля: 0.0%
date_2025-11-02 00:00:00
    пропусков: 0
    доля: 0.0%
date_2025-11-03 00:00:00
    пропусков: 0
    доля: 0.0%
date_2025-11-04 00:00:00
    пропусков: 0
    доля: 0.0%
date_2025-11-05 00:00:00
    пропусков: 0
    доля: 0.0%
date_2025-11-06 00:00:00
    пропусков: 0
    доля: 0.0%
date_2025-11-07 00:00:00
    пропусков: 0
    доля: 0.0%
date_2025-11-08 00:00:00
    пропусков: 0
    доля: 0.0%
date_2025-11-09 00:00:00
    пропусков: 0
    доля: 0.0%
date_2025-11-10 00:00:00
    пропусков: 0
    доля: 0.0%
date_2025-11-11 00:00:00
    пропусков: 0
    доля: 0.0%
date_2025-11-12 00:00:00
    пропусков: 0
    доля: 0.0%
date_2025-11-13 00:00:00
    пропусков: 0
    доля: 0.0%
date_2025-11-14 00:00:00
    пропусков: 0
    доля: 0.0%
ads_activity
    пропусков: 233
    доля: 4.0%
surf_depth
    пропусков: 111
    доля: 1.91%
primary_device
    пропусков: 157
    доля: 2.69%
cloud_usage
    пропусков: 146
    доля: 2.51%
```

Проверим сколько значений теряется при удалении

```
In [17]: df_clean = df.dropna()

print("Было:", len(df))
```

```
print("Стало:", len(df_clean))
print("Потеря:", len(df) - len(df_clean))
print("Доля потерь:", (len(df) - len(df_clean)) / len(df))
```

Out:

```
Было: 5826
Стало: 5205
Потеря: 621
Доля потерь: 0.10659114315139032
```

Потери в случае удаления существенны, удаление приводит к уменьшению объема более 10%

Вывод

Удаление строк с пропусками нецелесообразно необходимо сохранять данные и обрабатывать пропуски отдельно

Обработка дубликатов

```
In [18]: def duplicates_report(df, id_col='user_id'):
        full_dup = df.duplicated().sum()
        print("Полные дубликаты строк:", full_dup)

        id_dup = df.duplicated(subset=[id_col]).sum()
        print("Дубликаты по id:", id_dup)

        no_id = df.drop(columns=[id_col])
        no_id_dup = no_id.duplicated().sum()
        print("Дубликаты без id:", no_id_dup)
        duplicates_report(df)
```

Out:

```
Полные дубликаты строк: 0
Дубликаты по id: 0
Дубликаты без id: 0
```

Дубликатов нет

Анализ категориальных признаков

- Сколько уникальных значений в каждом категориальном признаке.
- Какие признаки можно кодировать One-Hot Encoding, а какие требуют специальных методов из-за высокой кардинальности.

```
In [19]: def categorical_analysis(df, cat_cols):
        for col in cat_cols:
            print("Столбец:", col)
            print("Уникальных значений:", df[col].nunique())
            print("Примеры значений:", df[col].unique()[:5])
            print()
```

```
In [20]: cat_cols = ['ads_activity', 'surf_depth', 'primary_device']
        categorical_analysis(df, cat_cols)
```

Out:

```
Столбец: ads_activity
Уникальных значений: 5
Примеры значений: <StringArray>
[nan, 'умеренно', 'редко', 'очень редко', 'очень часто']
Length: 5, dtype: str

Столбец: surf_depth
Уникальных значений: 3
Примеры значений: <StringArray>
```

```
['глубоко', 'средне', 'поверхностно', nan]
Length: 4, dtype: str

Столбец: primary_device
Уникальных значений: 4
Примеры значений: <StringArray>
['смартфон', 'ноутбук', 'ПК', 'планшет', nan]
Length: 5, dtype: str
```

Вывод

все признаки обладают небольшой кардинальностью для всех подходит ONE

Анализ выбросов и распределения

```
In [21]: def data_review(df, col):
          data = df[col].dropna()
          print(data.describe())

          q1 = data.quantile(0.25)
          q3 = data.quantile(0.75)
          iqr = q3 - q1
          lower_bound = q1 - 1.5 * iqr
          upper_bound = q3 + 1.5 * iqr

          outliers = data[(data < lower_bound) | (data > upper_bound)]
          high_outliers = data[data > upper_bound].sort_values(ascending=True)

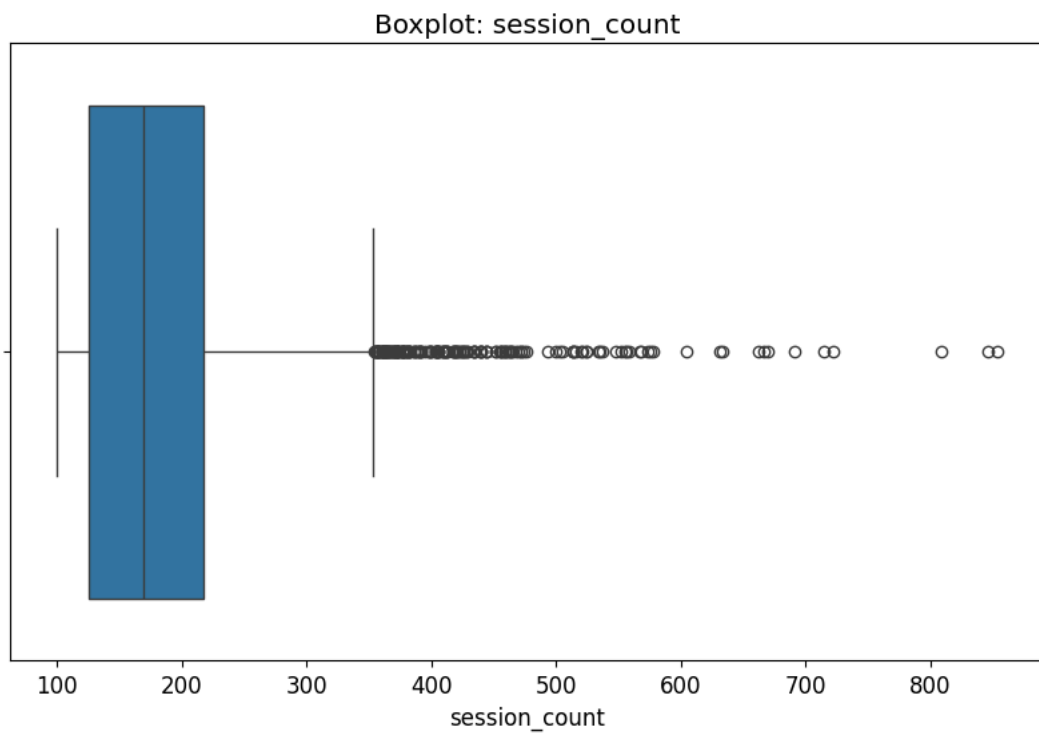
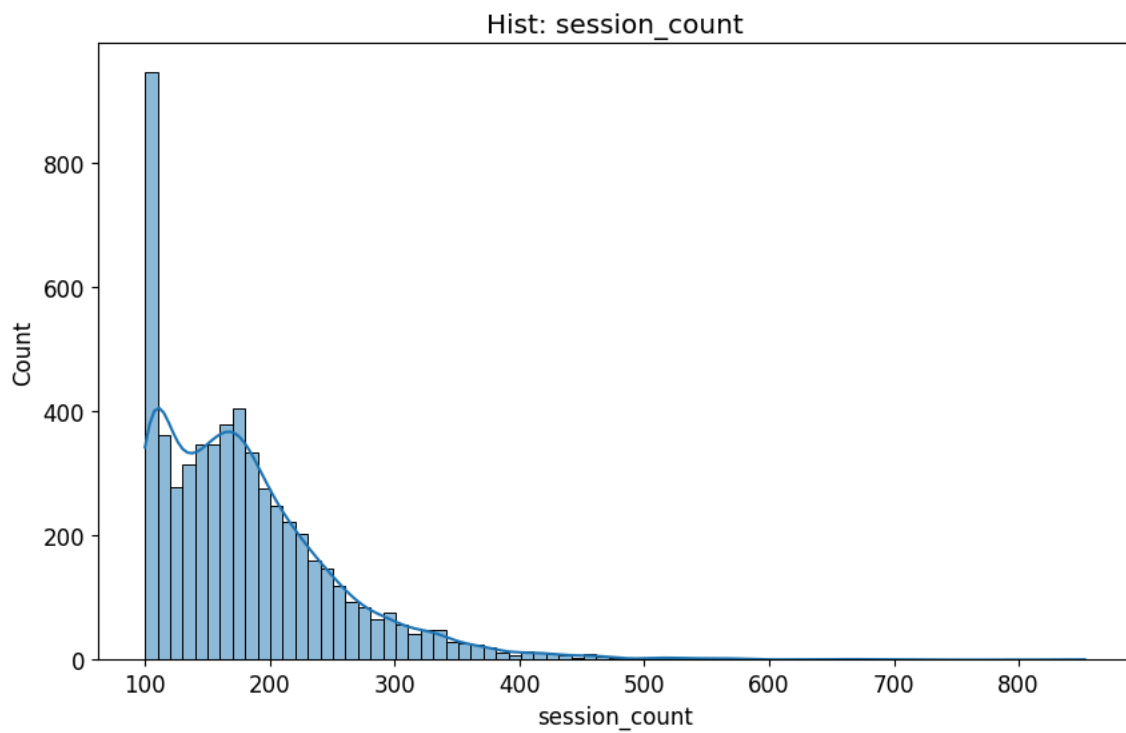
          print("Выбросы:")
          print("Количество:", len(outliers))
          print("Процент:", round(len(outliers) / len(data) * 100, 2))
          print("Границы:", round(lower_bound, 2), round(upper_bound, 2))
          print("Первые 5 выбросов выше верхней границы:", high_outliers.head(5).values)
          print("5 максимальных значений:", high_outliers.tail(5).values)

          sns.histplot(data, kde=True)
          plt.title(f"Hist: {col}")
          plt.show()

          sns.boxplot(x=data)
          plt.title(f"Boxplot: {col}")
          plt.show()
          data_review(df, 'session_count')
```

Out:

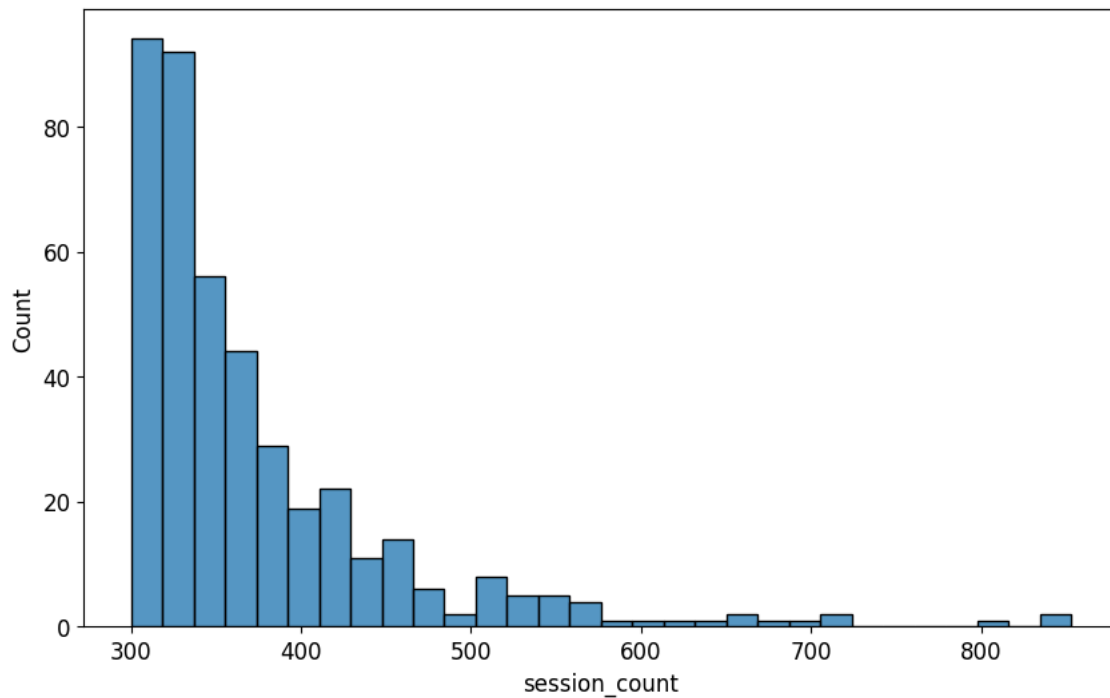
```
count    5826.000000
mean      182.929111
std        76.390669
min       100.000000
25%       126.000000
50%       169.000000
75%       217.000000
max       853.000000
Name: session_count, dtype: float64
Выбросы:
Количество: 187
Процент: 3.21
Границы: -10.5 353.5
Первые 5 выбросов выше верхней границы: [354 355 355 355 355]
5 максимальных значений: [714 722 808 846 853]
```



Довольно большое количество пользователей с выбросами, однако это могут быть пользователи просто с высокой активностью проверим

```
In [22]: high_activity = df[df['session_count'] >= 300]['session_count']
sns.histplot(high_activity, bins=30)
plt.show()
```


Out:



```
In [23]: for i in range(300, 851, 50):
          count = (df['session_count'] >= i).sum()
          print(f"{count} пользователей с активностью выше {i}")
```

Out:

```
424 пользователей с активностью выше 300
197 пользователей с активностью выше 350
104 пользователей с активностью выше 400
57 пользователей с активностью выше 450
36 пользователей с активностью выше 500
21 пользователей с активностью выше 550
12 пользователей с активностью выше 600
9 пользователей с активностью выше 650
5 пользователей с активностью выше 700
3 пользователей с активностью выше 750
3 пользователей с активностью выше 800
1 пользователей с активностью выше 850
```

Вывод высокоактивных пользователей

- Длинный правый хвост распределения по активности
- 424 пользователя имеют активность выше 300 сессий
- Высокие значения (500+) встречаются редко

Количество пользователей равномерно уменьшается, но из-за длины правого хвоста необходимо провести логарифмирование для более равномерного распределения

Корреляции¶

```
In [24]: def phik_corr_matrix(df, interval_cols=None, drop_cols=None):
          data = df.copy()

          if drop_cols:
              data = data.drop(columns=drop_cols, errors='ignore')

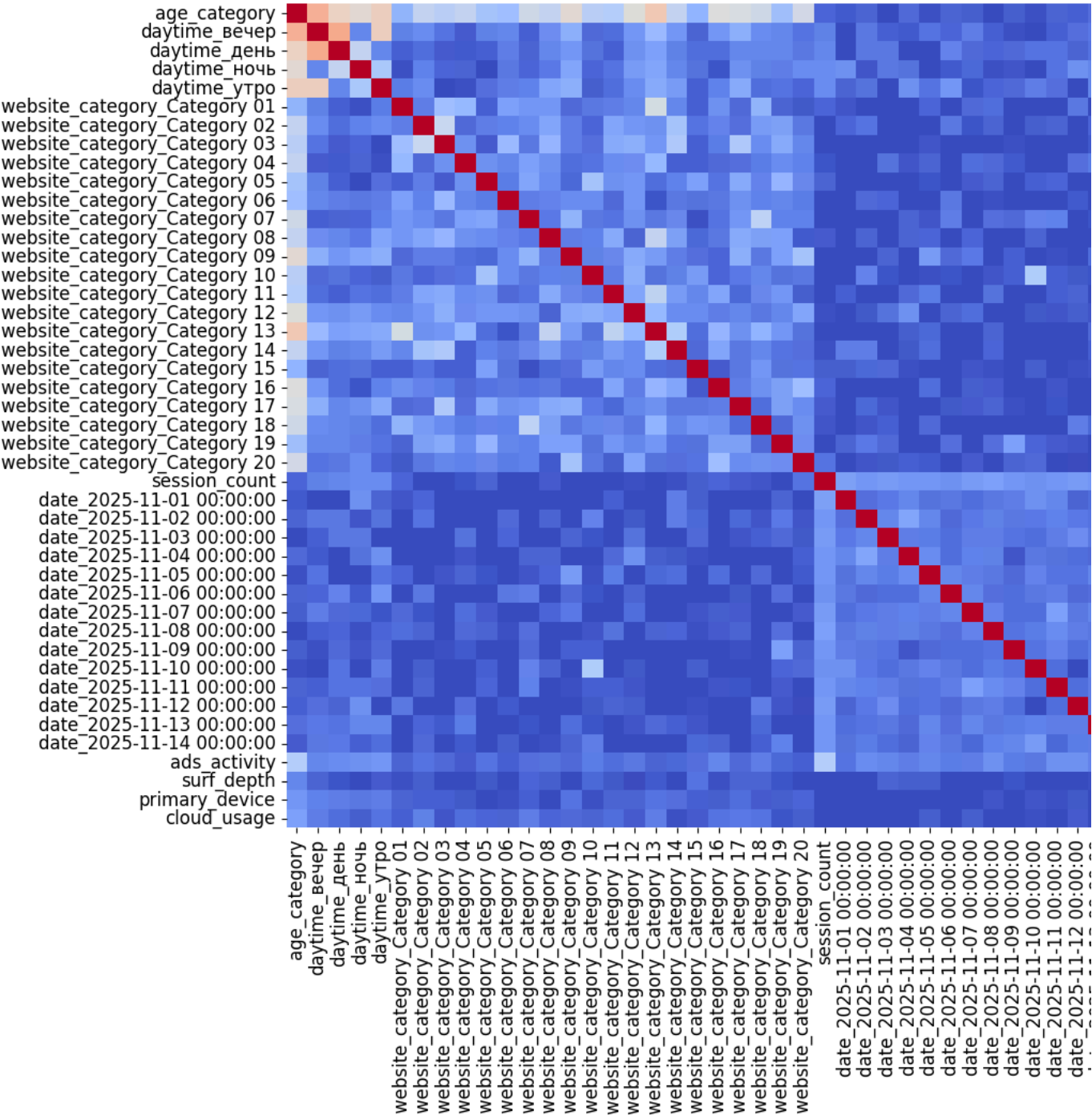
          corr = data.phik_matrix(interval_cols=interval_cols)

          plt.figure(figsize=(14, 10))
          sns.heatmap(corr, cmap='coolwarm')
          plt.show()
```

```
return corr
```

```
In [25]: drop_cols = ['user_id']
interval_cols = [x for x in df.columns if df[x].dtype in ["int64", "float64"]]
phik_corr_matrix(df=df, interval_cols=interval_cols, drop_cols=drop_cols)
```

Out:



	age_category	daytime_вечер	daytime_день	daytime_ночь	daytime_утро	website_category_Category 01	website
age_category	1.000000	0.682386	0.569928	0.525390	0.576593	0.266043	0.40988
daytime_вечер	0.682386	1.000000	0.700365	0.147026	0.574051	0.126842	0.15784
daytime_день	0.569928	0.700365	1.000000	0.416402	0.144613	0.049937	0.07951
daytime_ночь	0.525390	0.147026	0.416402	1.000000	0.343859	0.015393	0.13217
daytime_утро	0.576593	0.574051	0.144613	0.343859	1.000000	0.113834	0.15401

	age_category	daytime_вечер	daytime_день	daytime_ночь	daytime_утро	website_category_Category 01	website
website_category_Category 01	0.266043	0.126842	0.049937	0.015393	0.113834	1.000000	0.12063
website_category_Category 02	0.409882	0.157849	0.079515	0.132178	0.154014	0.120638	1.00000
website_category_Category 03	0.378157	0.097162	0.040807	0.063826	0.000000	0.309363	0.42583
website_category_Category 04	0.416316	0.039093	0.037035	0.056372	0.011796	0.281648	0.07036
website_category_Category 05	0.323442	0.127550	0.051894	0.000000	0.086384	0.048531	0.12390
website_category_Category 06	0.310331	0.138614	0.117770	0.107581	0.124796	0.211933	0.12957
website_category_Category 07	0.456138	0.049993	0.062454	0.066401	0.130448	0.189487	0.15636
website_category_Category 08	0.411383	0.162110	0.136877	0.111477	0.221030	0.182669	0.22331
website_category_Category 09	0.530918	0.254526	0.187753	0.223222	0.258583	0.076704	0.12132
website_category_Category 10	0.381601	0.071522	0.091541	0.063931	0.044153	0.104428	0.06313
website_category_Category 11	0.366538	0.083863	0.048389	0.094654	0.079241	0.117892	0.22424
website_category_Category 12	0.504003	0.188895	0.170544	0.197521	0.152439	0.142713	0.17847
website_category_Category 13	0.595197	0.292650	0.218364	0.218971	0.244942	0.474081	0.16679
website_category_Category 14	0.411280	0.190324	0.126691	0.144438	0.199433	0.147443	0.32935
website_category_Category 15	0.269583	0.056028	0.082084	0.091660	0.048259	0.049421	0.09948
website_category_Category 16	0.503536	0.177744	0.078966	0.020315	0.129486	0.168721	0.13657
website_category_Category 17	0.478876	0.247836	0.136861	0.174570	0.239212	0.075944	0.08922
website_category_Category 18	0.451252	0.158891	0.145097	0.127207	0.078329	0.266508	0.19331
website_category_Category 19	0.316357	0.143544	0.141080	0.118947	0.094314	0.054780	0.20695
website_category_Category 20	0.463448	0.096821	0.102550	0.154600	0.065521	0.032602	0.10357
session_count	0.065577	0.128174	0.148245	0.152359	0.156182	0.066128	0.00000
date_2025-11-01 00:00:00	0.038890	0.000000	0.000000	0.174729	0.059448	0.029394	0.06461
date_2025-11-02 00:00:00	0.057096	0.107281	0.106985	0.024006	0.093421	0.000000	0.09391
date_2025-11-03 00:00:00	0.000000	0.049964	0.107994	0.109024	0.000000	0.000000	0.00000
date_2025-11-04 00:00:00	0.075516	0.037260	0.000000	0.090482	0.168585	0.000000	0.00000
date_2025-11-05 00:00:00	0.016501	0.096876	0.000000	0.000000	0.098294	0.065123	0.00000
date_2025-11-06 00:00:00	0.060707	0.063562	0.135608	0.037083	0.190133	0.000000	0.10655
date_2025-11-07 00:00:00	0.046544	0.122043	0.075033	0.068295	0.022686	0.071454	0.02213
date_2025-11-08 00:00:00	0.000000	0.048739	0.066237	0.011785	0.062622	0.085374	0.05605

	age_category	daytime_вечер	daytime_день	daytime_ночь	daytime_утро	website_category_Category 01	website
date_2025-11-09 00:00:00	0.046734	0.016830	0.076932	0.029666	0.076937	0.000000	0.00000
date_2025-11-10 00:00:00	0.012879	0.000000	0.097731	0.000000	0.122592	0.000000	0.04316
date_2025-11-11 00:00:00	0.054768	0.048624	0.099381	0.094263	0.087835	0.000000	0.05952
date_2025-11-12 00:00:00	0.048132	0.089019	0.062155	0.174362	0.000000	0.105666	0.00000
date_2025-11-13 00:00:00	0.087176	0.112586	0.098299	0.144520	0.146821	0.000000	0.06518
date_2025-11-14 00:00:00	0.041746	0.105411	0.109916	0.051745	0.081480	0.000000	0.07066
ads_activity	0.367874	0.146063	0.154761	0.171146	0.183831	0.060967	0.10065
surf_depth	0.140736	0.058999	0.023306	0.000000	0.042699	0.009040	0.04666
primary_device	0.183185	0.132313	0.121429	0.108342	0.120165	0.058923	0.05242
cloud_usage	0.214420	0.120720	0.085472	0.045328	0.095722	0.051590	0.12774

44 rows × 44 columns

1. Связь признаков с целевой переменной (age_category)

- Сильные зависимости (0.5+):
 - daytime_вечер - 0.68
 - daytime_утро - 0.57
 - daytime_день - 0.56
 - website_category_Category 09 - 0.53
 - website_category_Category 13 - 0.59
- Средние зависимости (0.3–0.5):
 - ads_activity - 0.37
 - многие website_category_* - 0.30–0.45
- Слабые зависимости (<0.2):
 - session_count - 0.07
 - surf_depth - 0.14
 - primary_device - 0.18
 - cloud_usage - 0.21

2. Вывод по признакам

Возраст сильнее всего связан с временем и категорией сайта

Этап 3 Предобработка: Собираем пайплайн. Делим данные 80/20 тестовую выборку не трогаем до самого конца.

```
In [26]: def split_data(df, target, drop_cols=None, test_size=0.2, random_state=42):
          if drop_cols is None:
              drop_cols = []
          X = df.drop(columns=[target] + drop_cols)
          y = df[target]

          X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=test_size, random_state=random_state, stratify=y)

          print("Размер обучающей выборки:", X_train.shape)
          print("Размер тестовой выборки:", X_test.shape)
          print("Распределение target в train:")
          print(y_train.value_counts(normalize=True))
          print("Распределение target в test:")
          print(y_test.value_counts(normalize=True))
```

```
return X_train, X_test, y_train, y_test
```

```
In [27]: X_train, X_test, y_train, y_test = split_data(df=df, target='age_category')
```

Out:

Размер обучающей выборки: (4660, 44)

Размер тестовой выборки: (1166, 44)

Распределение target в train:

age_category

4 0.303219

2 0.246996

3 0.215236

0 0.146352

1 0.088197

Name: proportion, dtype: float64

Распределение target в test:

age_category

4 0.302744

2 0.246998

3 0.215266

0 0.146655

1 0.088336

Name: proportion, dtype: float64

Данные успешно разделены с сохранением распределения целевой переменной в train / test

```
In [28]: def prepare_pipeline(num_cols, cat_cols, ordinal_cols=None, target_cols=None, missing_num="median",
                             scaling="standard", missing_cat='most_frequent', encode="onehot", collinear_cols=None):
    if collinear_cols is None:
        collinear_cols = []
    if ordinal_cols is None:
        ordinal_cols = []
    if target_cols is None:
        target_cols = []

    num_cols = [c for c in num_cols if c not in collinear_cols]
    cat_cols = [c for c in cat_cols if c not in collinear_cols]
    target_cols = [c for c in target_cols if c not in collinear_cols]

    num_steps = [("imputer", SimpleImputer(strategy=missing_num))]
    if scaling == "standard":
        num_steps.append(("scaler", StandardScaler()))
    elif scaling == "minmax":
        num_steps.append(("scaler", MinMaxScaler()))
    num_pipe = Pipeline(num_steps)

    cat_steps = [("imputer", SimpleImputer(strategy=missing_cat))]
    if encode == "onehot":
        cat_steps.append(("encoder", OneHotEncoder(handle_unknown="ignore", sparse_output=False)))
    cat_pipe = Pipeline(cat_steps)

    transformers = [("num", num_pipe, num_cols),
                    ("cat", cat_pipe, cat_cols),]

    for col, categories in ordinal_cols:
        ordinal_pipe = Pipeline([("imputer", SimpleImputer(strategy="most_frequent")),
                                  ("encoder", OrdinalEncoder(categories=[categories]))])

        transformers.append(("ord_"+col, ordinal_pipe, [col]))

    if target_cols:
        target_pipe = Pipeline([("encoder", TargetEncoder(handle_unknown="value", handle_missing="value"))])
        transformers.append(("target", target_pipe, target_cols))
```

```
preprocessor = ColumnTransformer(transformers)
return preprocessor
```

```
In [29]: num_cols = ['session_count']

pass_cols = [col for col in X_train.columns if ('daytime_' in col or 'website_category_' in col or 'date_' in col)]

cat_cols = ['ads_activity', 'surf_depth', 'primary_device', 'cloud_usage']

preprocessor = prepare_pipeline(
    num_cols=num_cols,
    cat_cols=cat_cols,
    missing_num='median',
    scaling='standard',
    missing_cat='most_frequent',
    encode='onehot'
)

transformers = preprocessor.transformers

preprocessor = ColumnTransformer(transformers=[*transformers, ('pass', 'passthrough', pass_cols)])

X_train_prepared = preprocessor.fit_transform(X_train.drop(columns=['user_id']), y_train)
X_test_prepared = preprocessor.transform(X_test.drop(columns=['user_id']))

print("Размер после предобработки:")
print("Train:", X_train_prepared.shape)
print("Test:", X_test_prepared.shape)
```

```
Out:
Размер после предобработки:
Train: (4660, 53)
Test: (1166, 53)
```

Подготовка данных и разбиение выборки

- Данные были разделены на обучающую и тестовую выборки с помощью `train_test_split`.
- Использовали `stratify=y`, благодаря чему распределение `age_category` сохранилось в train и test выборке.
- Был построен пайплайн предобработки данных
- Для числового признака `session_count`:
 - пропуски заполнялись медианой;
 - масштабирование `StandardScaler`.
- Для категориальных признаков: `ads_activity`, `surf_depth`, `primary_device`, `cloud_usage`:
 - заполнение пропусков наиболее частым значением;
 - One-Hot Encoding.
- После One-Hot Encoding количество признаков увеличилось, так как категориальные признаки были преобразованы в бинарные колоноки.
- Размеры итоговых датасетов после предобработки:
 - Train: (4660, 53)
 - Test: (1166, 53)

Этап 4 Отбор признаков: используем методы-обёртки для поиска оптимального набора признаков.

```
In [30]: def feature_select(X_train, y_train, preprocessor, methods, cols_num):
        cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
```

```

results = {}
feature_counts = [int(cols_num * 0.25), int(cols_num * 0.50), int(cols_num * 0.75), cols_num]

for method in methods:
    for k in feature_counts:
        if method == "kbest":
            pipe = Pipeline([
                ("preprocessor", preprocessor),
                ("selector", SelectKBest(score_func=mutual_info_classif, k=k)),
                ("model", LogisticRegression(max_iter=5000, random_state=42))
            ])

            elif method == "rfe":
                pipe = Pipeline([
                    ("preprocessor", preprocessor),
                    ("selector", RFE(estimator=LogisticRegression(max_iter=1000, random_state=42), n_features_to_select=k, step
=0.2)),
                    ("model", LogisticRegression(max_iter=5000, random_state=42))
                ])

            else:
                print("Метод", method, "не реализован")

            scores = cross_validate(pipe, X_train, y_train, cv=cv, scoring={"f1_macro": "f1_macro"}, n_jobs=-1)
            results[method, k] = scores["test_f1_macro"].mean()

            print(method)
            print("Количество признаков:", k)
            print("F1-macro:", scores["test_f1_macro"].mean())
            print()

    return results

```

In [31]: results = feature_select(X_train=X_train, y_train=y_train, preprocessor=preprocessor, methods=["kbest", "rfe"], cols_num=53)

Out:

```

kbest
Количество признаков: 13
F1-макро: 0.6100914784002096

kbest
Количество признаков: 26
F1-макро: 0.6704307395532066

kbest
Количество признаков: 39
F1-макро: 0.6758794229352654

kbest
Количество признаков: 53
F1-макро: 0.6742384124653469

rfe
Количество признаков: 13
F1-макро: 0.6155291908785978

rfe
Количество признаков: 26
F1-макро: 0.6725968868710612

rfe
Количество признаков: 39
F1-макро: 0.6679761143707988

rfe
Количество признаков: 53
F1-макро: 0.6742384124653469

```

Вывод по отбору признаков

- Лучший результат - SelectKBest при использовании 26 признаков: F1-macro = 0.676
- Улучшение при уменьшении числа признаков есть
- Нужно использовать отбор признаков SelectKBest на 26 признаков

Этап 5 Базовая модель: обучаем DummyClassifier и многоклассовые модели OvR, OvO, multinomial LogisticRegression модели.

```
In [32]: def evaluate_models(models, X, y, preprocessor, metrics, selector=None):
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
results = []
scoring = {}

for metric in metrics:
    scoring[metric] = metric

for name, model in models.items():
    steps = []
    if preprocessor is not None:
        steps.append(("preprocessor", preprocessor))
    if selector is not None:
        steps.append(("selector", selector))
    steps.append(("model", model))
    pipe = Pipeline(steps)
    scores = cross_validate(pipe, X, y, cv=cv, scoring=scoring, n_jobs=-1)
    result = {"model": name}
    for metric in metrics:
        result[metric] = scores["test_" + metric].mean()
    results.append(result)
    print(name)
    for metric in metrics:
        print(metric + ":", scores["test_" + metric].mean())
    print()
    pipe.fit(X, y)
    y_pred = pipe.predict(X)
    conf = confusion_matrix(y, y_pred)
    print(conf)
    sns.heatmap(conf, annot=True)
    plt.show()

return pd.DataFrame(results)

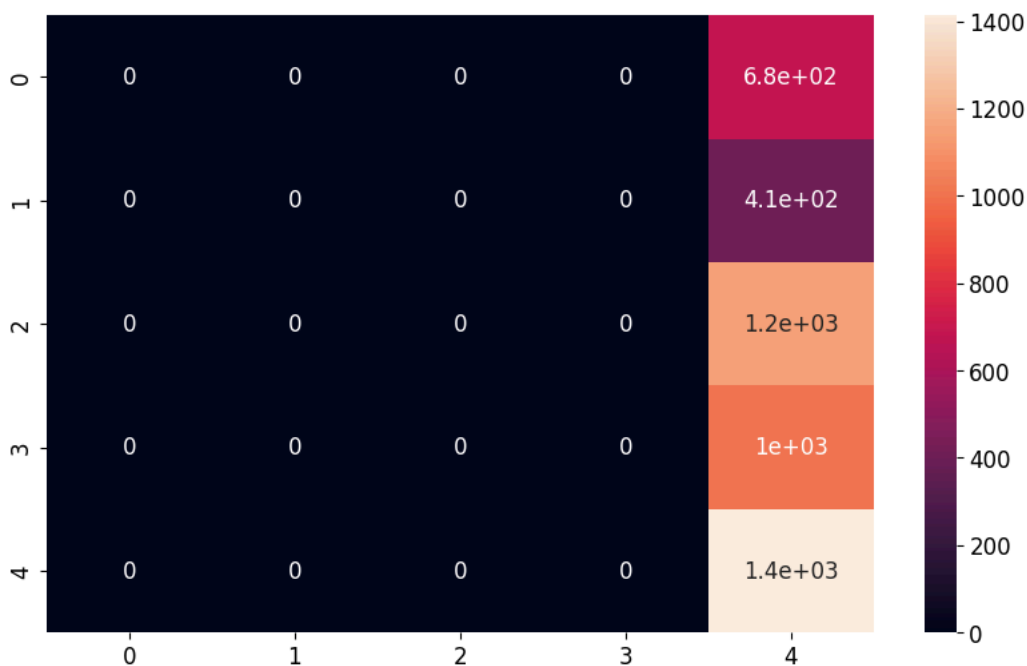
In [33]: models = {
    "Dummy": DummyClassifier(strategy="most_frequent"),
    "OvR LogisticRegression": OneVsRestClassifier(LogisticRegression(max_iter=1000, random_state=42)),
    "OvO LogisticRegression": OneVsOneClassifier(LogisticRegression(max_iter=1000, random_state=42)),
    "Multinomial LogisticRegression": LogisticRegression(max_iter=1000, random_state=42),
    "LinearSVC": LinearSVC(random_state=42),
    "SVC linear": SVC(kernel="linear", random_state=42),
    "SVC rbf": SVC(kernel="rbf", random_state=42),
    "SVC poly": SVC(kernel="poly", degree=5, random_state=42),
    "SVC sigmoid": SVC(kernel="sigmoid", random_state=42)
}
metrics = ["accuracy", "precision_macro", "recall_macro", "f1_macro"]
results = evaluate_models(models, X_train.drop(columns=['user_id']), y_train, preprocessor, metrics)
```

Out:

```
Dummy
accuracy: 0.3032188841201717
precision_macro: 0.06064377682403434
recall_macro: 0.2
f1_macro: 0.09306762666015822
```

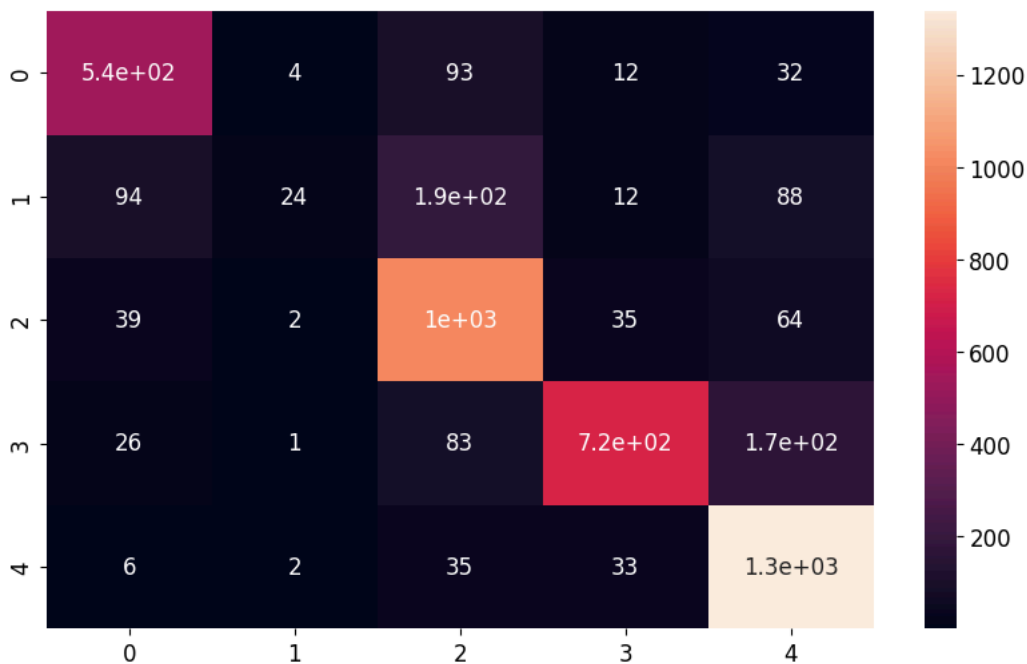


```
[[ 0  0  0  0 682]
 [ 0  0  0  0 411]
 [ 0  0  0  0 1151]
 [ 0  0  0  0 1003]
 [ 0  0  0  0 1413]]
```



OvR LogisticRegression
accuracy: 0.7609442060085837
precision_macro: 0.7375324210933253
recall_macro: 0.6569183175661675
f1_macro: 0.6402028468342128

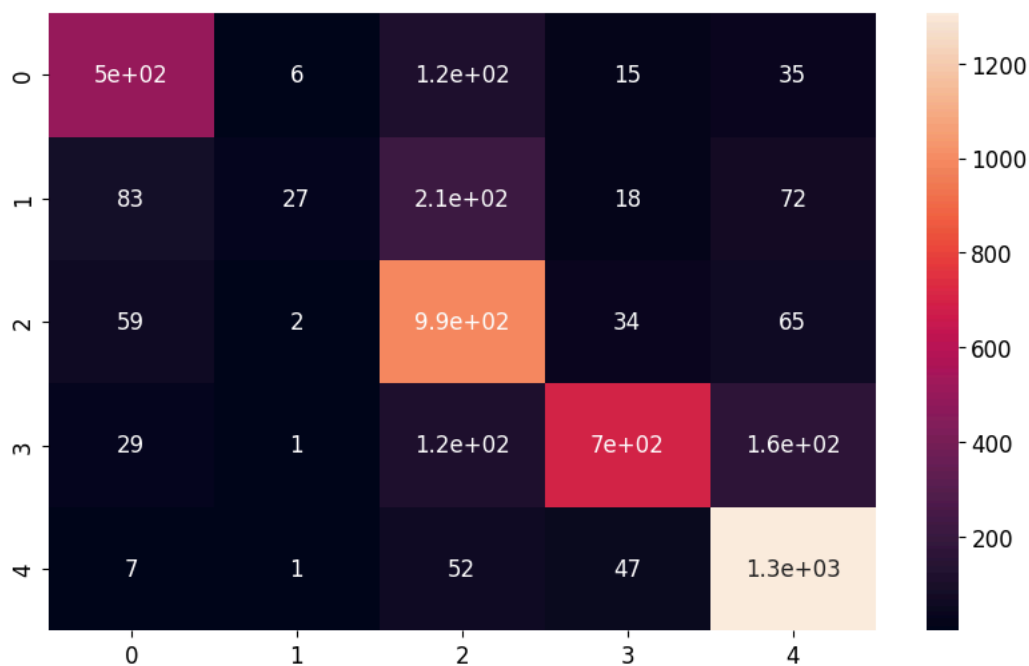
```
[[ 541  4  93  12  32]
 [ 94 24 193  12  88]
 [ 39  2 1011  35  64]
 [ 26  1  83 719 174]
 [  6  2  35  33 1337]]
```



OvO LogisticRegression
accuracy: 0.7319742489270386
precision_macro: 0.6790137366711445

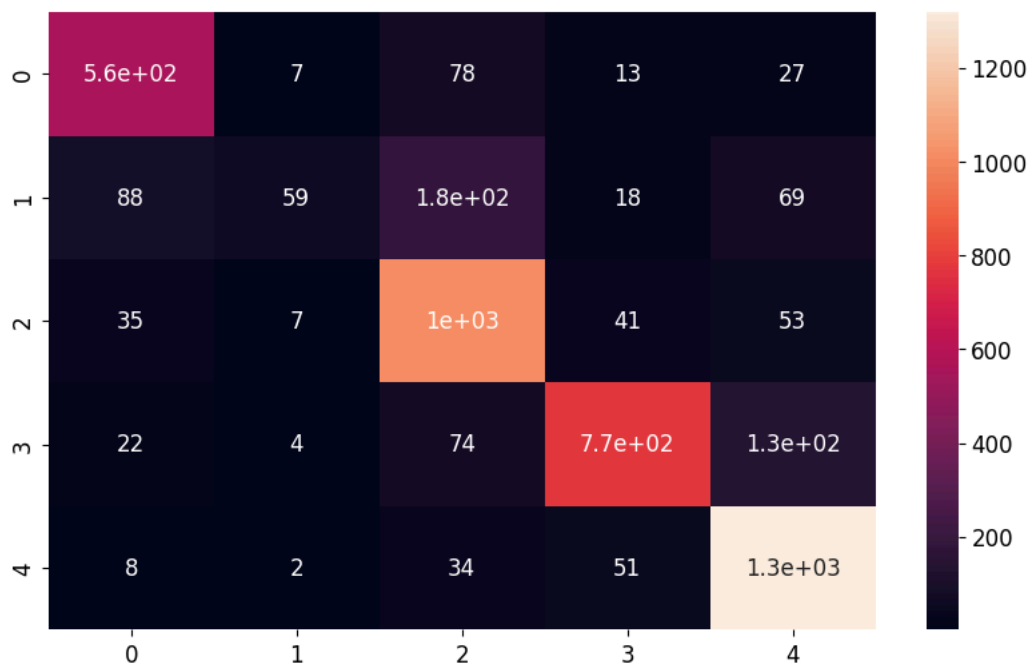
```
recall_macro: 0.6283249696034794
f1_macro: 0.6104644446820922
```

```
[[ 503   6  123   15   35]
 [  83  27  211   18   72]
 [  59   2  991   34   65]
 [  29   1  118  698  157]
 [   7   1   52  47 1306]]
```



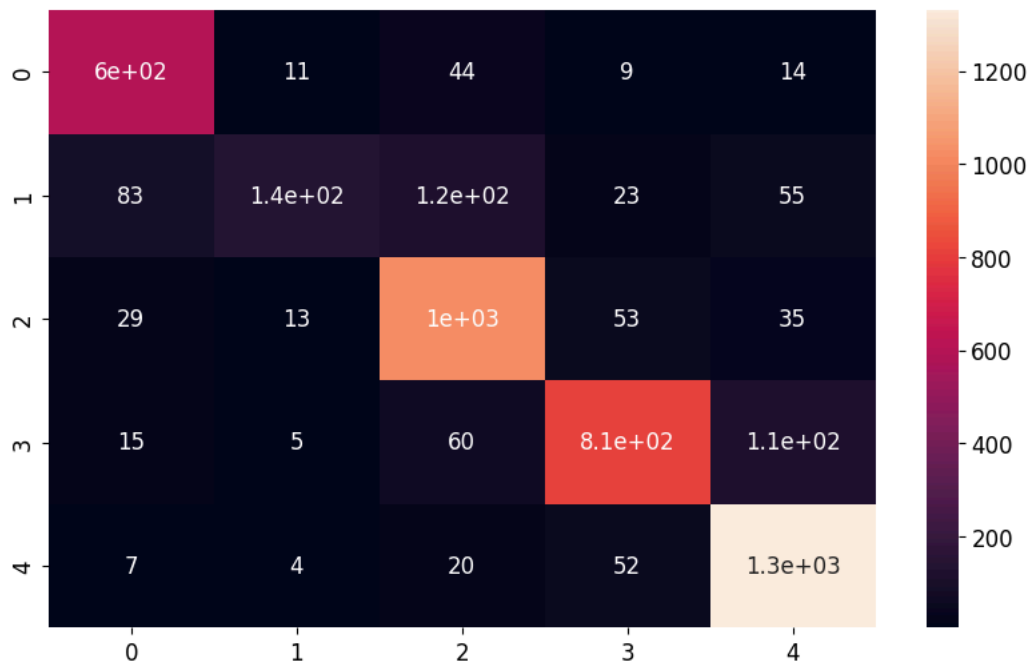
```
Multinomial LogisticRegression
accuracy: 0.7804721030042918
precision_macro: 0.7590870733800837
recall_macro: 0.6838667446580463
f1_macro: 0.6742384124653469
```

```
[[ 557   7   78   13   27]
 [  88  59  177   18   69]
 [  35   7 1015   41   53]
 [  22   4   74  774  129]
 [   8   2   34  51 1318]]
```



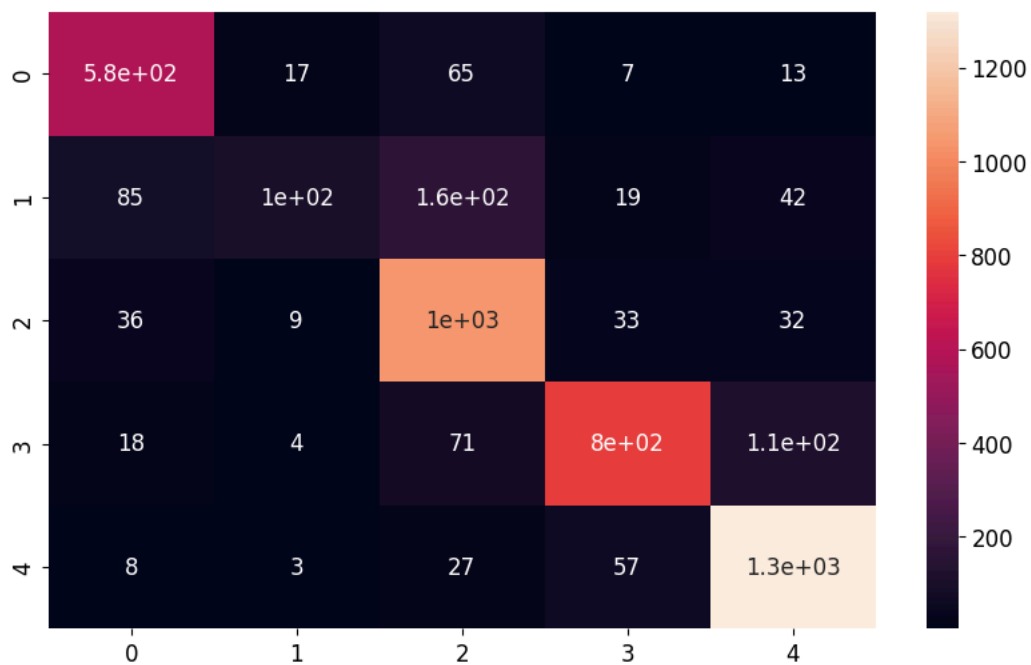
LinearSVC
accuracy: 0.8238197424892704
precision_macro: 0.8144233337053194
recall_macro: 0.753067739747624
f1_macro: 0.758315541006221

```
[[ 604  11  44   9  14]
 [  83 135 115  23  55]
 [  29  13 1021  53  35]
 [  15   5  60 809 114]
 [   7   4  20  52 1330]]
```



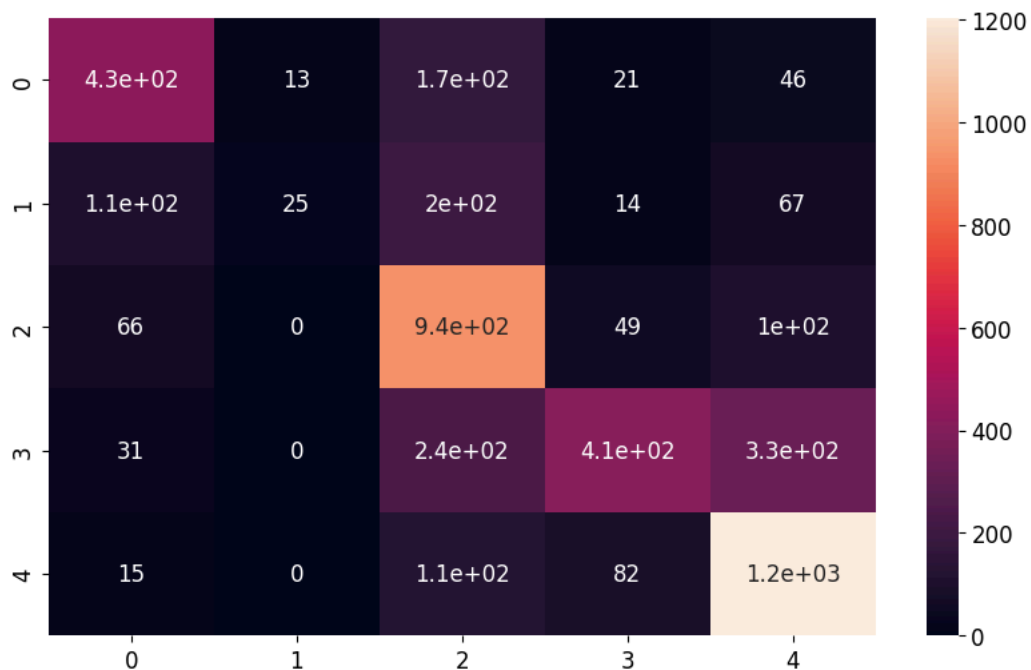
SVC linear
accuracy: 0.8042918454935621
precision_macro: 0.7918604119993384
recall_macro: 0.7161626750105445
f1_macro: 0.7114497975612792

```
[[ 580  17  65   7  13]
 [  85 102 163  19  42]
 [  36   9 1041  33  32]
 [  18   4  71 797 113]
 [   8   3  27  57 1318]]
```



SVC rbf
accuracy: 0.5626609442060085
precision_macro: 0.5612162998796025
recall_macro: 0.4767773811473132
f1_macro: 0.45734102601554366

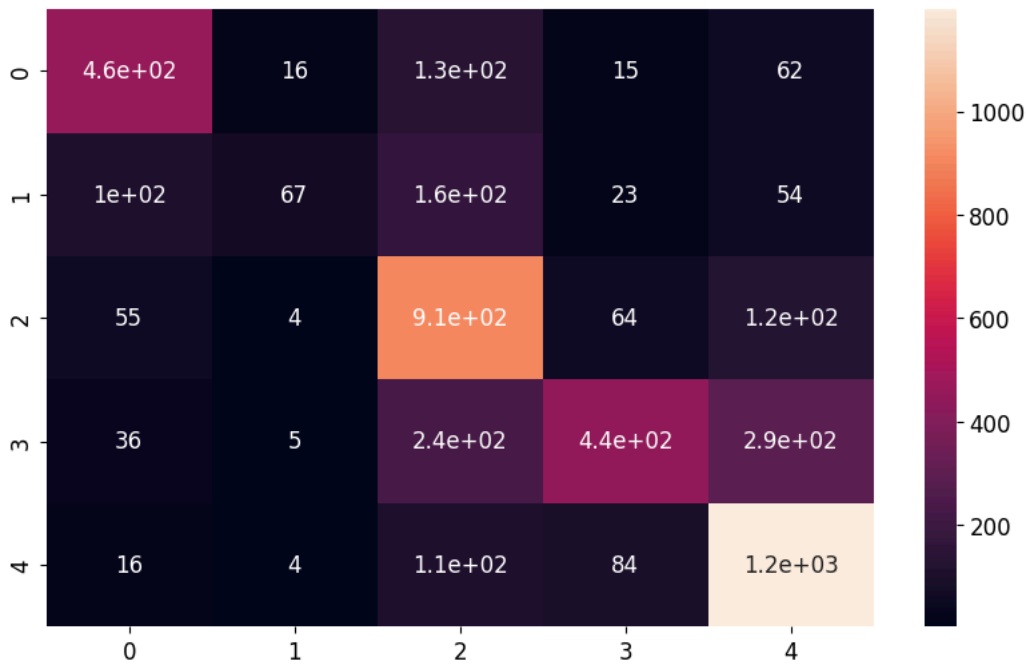
```
[[ 429  13  173  21  46]
 [ 107  25  198  14  67]
 [  66   0  935  49 101]
 [  31   0  236 409 327]
 [  15   0  114  82 1202]]
```



SVC poly
accuracy: 0.5418454935622317
precision_macro: 0.516044336431122
recall_macro: 0.4713202246906885
f1_macro: 0.46256201502423977

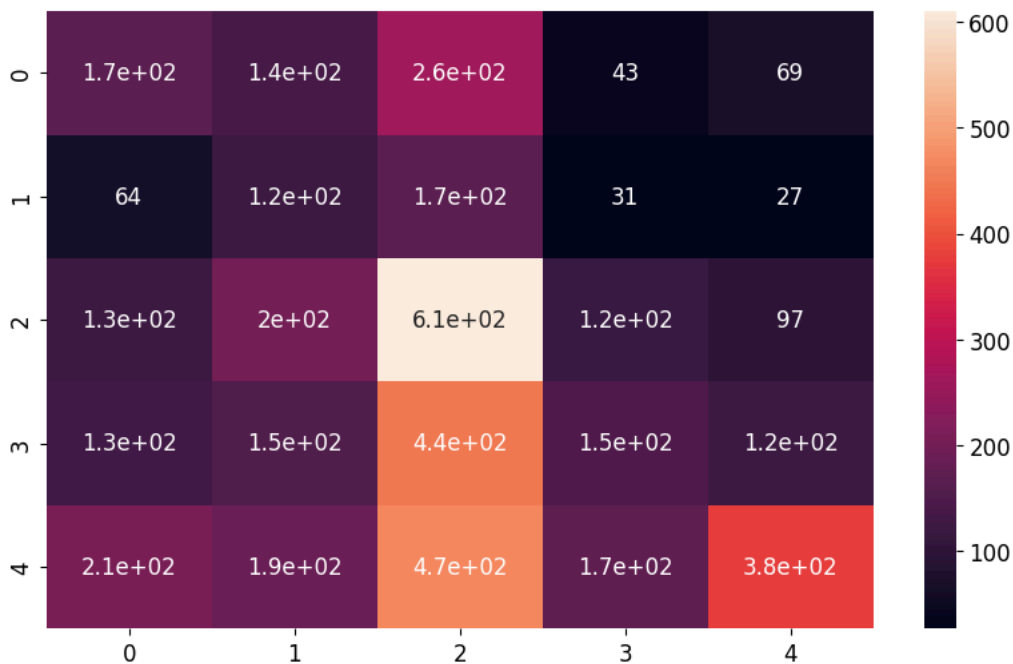
```
[[ 459  16  130  15  62]
 [ 103  67  164  23  54]
 [  55   4  909  64 119]]
```

```
[ 36  5 235 438 289]
[ 16  4 112  84 1197]]
```



```
SVC sigmoid
accuracy: 0.3109442060085837
precision_macro: 0.3012869533773498
recall_macro: 0.2960812201837579
f1_macro: 0.2769810107386475
```

```
[[167 138 265  43  69]
 [ 64 122 167  31  27]
 [127 200 610 117  97]
 [132 153 445 149 124]
 [211 187 467 171 377]]
```



Вывод

- Среди моделей LogisticRegression лучшая multinomial LogisticRegression:
 - Accuracy = 0.780 - модель правильно определяет класс у 78% объектов
 - Precision-макро = 0.76 у модели не много ложных срабатываний

- Recall-macro = 0.684 модель находит 68% объектов каждого класса
 - F1-macro = 0.675 баланс между precision и recall лучший среди логистических моделей
 - OvR и OvO LogisticRegression дают результат хуже по всем метрикам - менее стабильное разделение классов
- Среди моделей SVM лучшая LinearSVC:
 - Accuracy = 0.824 - модель правильно определяет класс у 82% объектов
 - Precision-macro = 0.814 - у модели не много ложных срабатываний
 - Recall-macro = 0.753 - модель находит 75% объектов каждого класса
 - F1-macro = 0.759 - лучший баланс между precision и recall среди всех моделей

SVC с линейным ядром показал близкие к LinearSVC результаты:

- Accuracy = 0.804
- F1-macro = 0.711

SVC с ядрами rbf, poly и sigmoid показали результаты более слабые чем остальные модели:

- RBF и polynomial ядра начали хуже разделять классы и допустили большое количество ошибок
- Sigmoid ядро показало результат accuracy, близкий к DummyClassifier, что означает непригодность модели для текущей задачи

- После ONE-преобразования данные стали близки к линейному разделению, поэтому линейные модели оказались эффективнее
- Лучшей стала модель LinearSVC
- Для данной задачи особенно важен recall несовершеннолетних(класс 0), так как ошибка, при которой несовершеннолетний пользователь определяется как совершеннолетний, является наиболее грубой в задаче, поэтому в дальнейшем необходимо изменить для повышение recall младших возрастных классов, даже если это приведёт к снижению precision.

Этап 6 Новые признаки: генерируем дополнительные признаки на основе уже имеющихся, проверяем помогают ли они модели.

Создадим новые признаки и проверим как они повлияют на метрики лучших моделей:

- weekend_activity - суммарная доля активности в выходные дни
- avg_day_activity - session_count деленное на количество дней в которых была активность
- category_time_prefer - какая категория используется во время суток основной активности
- category_use - session_count умноженный на долю по категории сайта
- daytime_use - session_count умноженный на долю по времени суток

```
In [34]: def add_features(X, visits_df=None):
    if visits_df is None:
        visits_df = visits
    X = X.copy()
    date_cols = [col for col in X.columns if "date_" in col]

    weekend_days = [col for col in date_cols if pd.to_datetime(col.replace("date_", "")).dayofweek in [5, 6]]
    X["weekend_activity"] = 0
    for col in weekend_days:
        X["weekend_activity"] += X[col]

    X["active_days"] = (X[date_cols] > 0).sum(axis=1)
    X["avg_day_activity"] = X["session_count"] / X["active_days"]
    daytime_cols = ["daytime_утро", "daytime_день", "daytime_вечер", "daytime_ночь"]
    X["main_daytime"] = (X[daytime_cols].idxmax(axis=1).str.replace("daytime_", ""))
    visits_temp = visits_df.merge(X[["user_id", "main_daytime"]], on="user_id", how="left")
    visits_temp = visits_temp[visits_temp["daytime"] == visits_temp["main_daytime"]]
    category_time_prefer = (visits_temp.groupby(["user_id", "website_category"]).size().unstack(fill_value=0).idxmax(axis=1).reset_index(name="category_time_prefer"))
    X = X.merge(category_time_prefer, on="user_id", how="left")
    website_category_cols = [col for col in X.columns if "website_category_" in col]
    for col in website_category_cols:
        X[col + "_use"] = X["session_count"] * X[col]

    daytime_cols = [col for col in X.columns if "daytime_" in col]
    for col in daytime_cols:
        X[col + "_use"] = X["session_count"] * X[col]
```

```
return X
```

```
In [35]: X_train_fe = add_features(X_train)
X_test_fe = add_features(X_test)

num_cols = ["session_count", "weekend_activity", "avg_day_activity"]

pass_cols = [col for col in X_train_fe.columns if ("daytime_" in col or
                                                    "website_category_" in col or
                                                    "date_" in col or
                                                    "_use" in col)]

cat_cols = ["ads_activity", "surf_depth", "primary_device", "cloud_usage", "category_time_prefer"]

preprocessor = prepare_pipeline(
    num_cols=num_cols,
    cat_cols=cat_cols,
    missing_num='median',
    scaling='standard',
    missing_cat='most_frequent',
    encode='onehot'
)

transformers = preprocessor.transformers

preprocessor = ColumnTransformer(transformers=[*transformers, ('pass', 'passthrough', pass_cols)])

X_train_prepared = preprocessor.fit_transform(X_train_fe, y_train)
X_test_prepared = preprocessor.transform(X_test_fe)

print("Размер после предобработки:")
print("Train:", X_train_prepared.shape)
print("Test:", X_test_prepared.shape)

results = feature_select(X_train=X_train_fe.drop(columns=['user_id']), y_train=y_train, preprocessor=preprocessor, methods=["kbest"], cols_num=99)
```

Out:

```
Размер после предобработки:
Train: (4660, 99)
Test: (1166, 99)
kbest
Количество признаков: 24
F1-макро: 0.6706537435055759

kbest
Количество признаков: 49
F1-макро: 0.8028701937278498

kbest
Количество признаков: 74
F1-макро: 0.8110303678695517

kbest
Количество признаков: 99
F1-макро: 0.8108703309772812
```

- После добавления новых признаков размерность увеличилась до 99
- По результатам отбора признаков видно, что не все признаки одинаково полезны для модели.
- Метод `kbest` показывает, что качество модели растёт при увеличении числа признаков, но после 74 признаков улучшение практически прекращается (F1-макро стабилизируется около 0.81).
- В данных есть часть избыточных признаков, которые не дают дополнительного прироста качества.
- Наиболее устойчивое качество достигается при использовании примерно 70–100% признаков

- В дальнейшем имеет смысл использовать метод отбора признаков kbest с 74 признаками

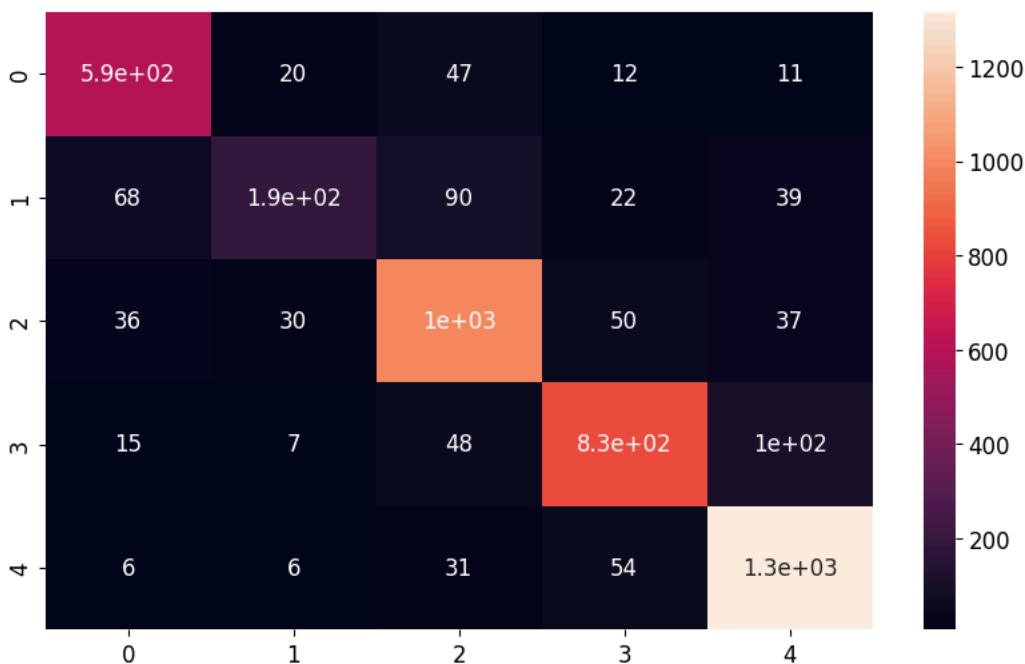
```
In [36]: models = {
    "LinearSVC": LinearSVC(),
    "Multinomial LogisticRegression": LogisticRegression(max_iter=1000, random_state=42),
}
```

```
In [37]: selector = SelectKBest(score_func=mutual_info_classif, k=74)
    results = evaluate_models(models, X_train_fe, y_train, preprocessor, ["accuracy", "precision_macro", "recall_macro", "f1_macro"], selector=selector)
```

Out:

```
LinearSVC
accuracy: 0.832618025751073
precision_macro: 0.8139492054872998
recall_macro: 0.783036827638435
f1_macro: 0.7921360590300683
```

```
[[ 592  20  47  12  11]
 [  68 192  90  22  39]
 [  36  30 998  50  37]
 [  15   7  48 829 104]
 [   6   6  31  54 1316]]
```



```
Multinomial LogisticRegression
accuracy: 0.8469957081545065
precision_macro: 0.8214659417375744
recall_macro: 0.8119697964706628
f1_macro: 0.8156176499578258
```

```
[[ 601  31  37   5   8]
 [  48 252  72  15  24]
 [  23  50 998  45  35]
 [   9  12  47 852  83]
 [   8  12  28  77 1288]]
```

C:\Users\user\AppData\Roaming\Python\Python313\site-packages\sklearn\linear_model_logistic.py:599: ConvergenceWarning: lbfgs failed to converge after 1000 iteration(s) (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=1000).

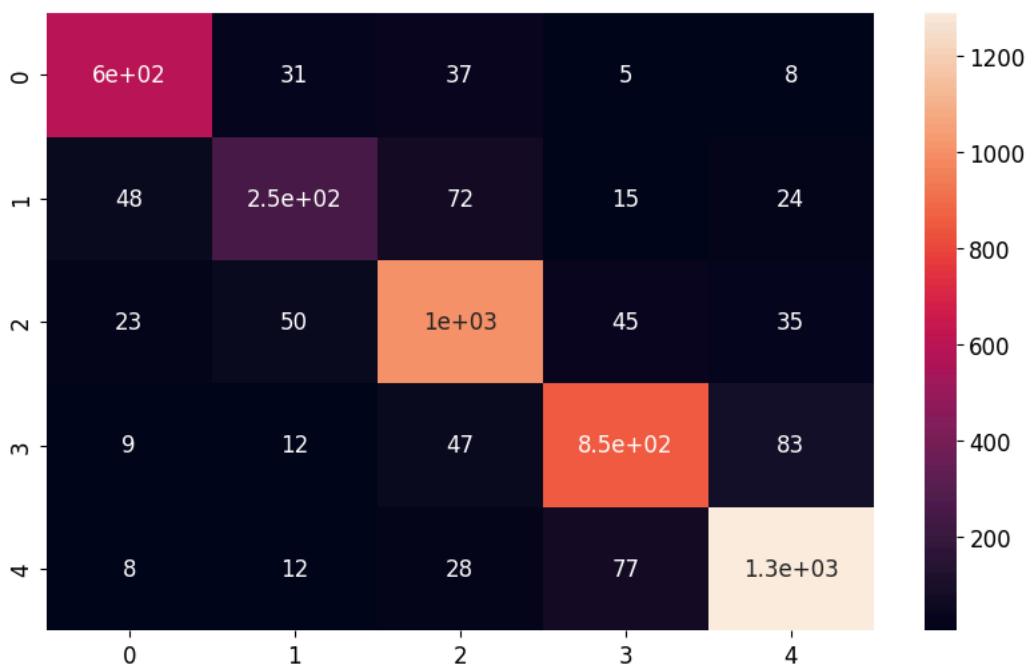
You might also want to scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(
```



Этап 7 Подбор гиперпараметров: перебираем параметры, сравниваем результаты в таблице, выбираем лучшую конфигурацию.

```
In [38]: cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

results = []

logreg_pipe = Pipeline([
    ("preprocessor", preprocessor),
    ("selector", SelectKBest(score_func=mutual_info_classif, k=74)),
    ("model", LogisticRegression(random_state=42, max_iter=1000))
])

logreg_grid = {
    "model__C": [0.01, 0.1, 1, 10],
    "model__class_weight": [None, "balanced"]
}

grid_logreg = GridSearchCV(
    estimator=logreg_pipe,
    param_grid=logreg_grid,
    cv=cv,
    scoring="f1_macro",
    n_jobs=-1,
    verbose=1
)

grid_logreg.fit(X_train_fe, y_train)

results.append(
    pd.DataFrame(grid_logreg.cv_results_).assign(model="LogReg")
)

svc_pipe = Pipeline([
    ("preprocessor", preprocessor),
    ("selector", SelectKBest(score_func=mutual_info_classif, k=74)),
    ("model", LinearSVC())
])
```

```

])

svc_grid = {
    "model__C": [0.01, 0.1, 1, 10],
    "model__class_weight": [None, "balanced"]
}

grid_svc = GridSearchCV(
    estimator=svc_pipe,
    param_grid=svc_grid,
    cv=cv,
    scoring="f1_macro",
    n_jobs=-1,
    verbose=1
)
grid_svc.fit(X_train_fe, y_train)

results.append(pd.DataFrame(grid_svc.cv_results_).assign(model="LinearSVC"))

all_models = pd.concat(results, ignore_index=True)

top10 = all_models.sort_values("mean_test_score", ascending=False).head(10)[["model", "params", "mean_test_score", "std_test_score", "rank_test_score"]]

pd.set_option("display.max_colwidth", None)
pd.set_option("display.width", None)
pd.set_option("display.max_columns", None)

print("\nTOP-10 моделей:")
display(top10)

```

Out:

Fitting 5 folds for each of 8 candidates, totalling 40 fits

C:\Users\user\AppData\Roaming\Python\Python313\site-packages\sklearn\linear_model_logistic.py:599: ConvergenceWarning: lbfgs failed to converge after 1000 iteration(s) (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=1000).

You might also want to scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

n_iter_i = _check_optimize_result(

Fitting 5 folds for each of 8 candidates, totalling 40 fits

TOP-10 моделей:

	model	params	mean_test_score	std_test_score	rank_test_score
6	LogReg	{'model__C': 10, 'model__class_weight': None}	0.815163	0.018009	1
4	LogReg	{'model__C': 1, 'model__class_weight': None}	0.813959	0.018536	2
2	LogReg	{'model__C': 0.1, 'model__class_weight': None}	0.813600	0.014870	3
0	LogReg	{'model__C': 0.01, 'model__class_weight': None}	0.811628	0.012981	4
3	LogReg	{'model__C': 0.1, 'model__class_weight': 'balanced'}	0.811239	0.021629	5
5	LogReg	{'model__C': 1, 'model__class_weight': 'balanced'}	0.811188	0.018121	6
1	LogReg	{'model__C': 0.01, 'model__class_weight': 'balanced'}	0.807505	0.017504	7
9	LinearSVC	{'model__C': 0.01, 'model__class_weight': 'balanced'}	0.807245	0.015039	1
7	LogReg	{'model__C': 10, 'model__class_weight': 'balanced'}	0.807043	0.018895	8

	model	params	mean_test_score	std_test_score	rank_test_score
11	LinearSVC	{'model_C': 0.1, 'model_class_weight': 'balanced'}	0.806375	0.019521	2

Из 10 лучших конфигураций 8 занимают модели LogisticRegression, что подтверждает её преимущество по качеству. В среднем она показывает более высокие значения метрик чем LinearSVC.

Далее проведём анализ влияния гиперпараметров на recall класса 0, так как в рамках задачи этот показатель является важным. При этом будем учитывать, чтобы улучшение recall не приводило к существенной потере по другим метрикам. Построим кривую precision-recall для задачи класс 0 или не 0

```
In [39]: pipe = Pipeline([("preprocessor", preprocessor),
                          ("selector", SelectKBest(score_func=mutual_info_classif, k=74)),
                          ("model", LogisticRegression(random_state=42, max_iter=10000))
                        ])

pipe.fit(X_train_fe, y_train)
y_proba = cross_val_predict(pipe, X_train_fe, y_train, cv=5, method="predict_proba", n_jobs=-1)
y_train_bin = (y_train == 0).astype(int)
y_proba_0 = y_proba[:, 0]

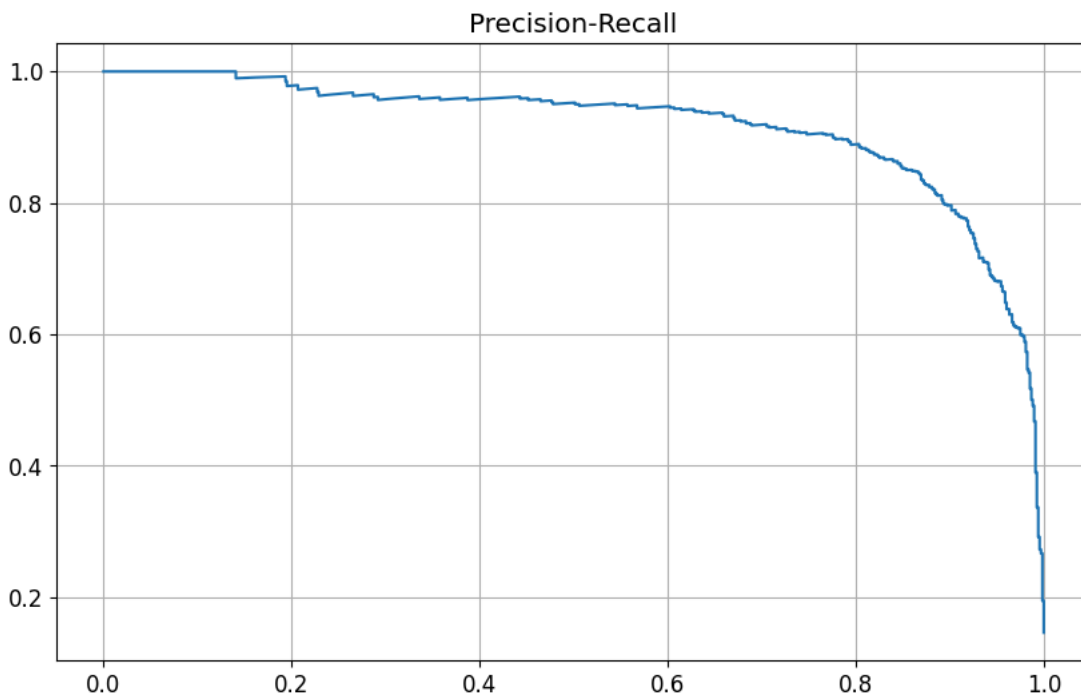
precision, recall, thresholds = precision_recall_curve(y_train_bin, y_proba_0)

plt.plot(recall, precision)

plt.title("Precision-Recall")

plt.grid()
plt.show()
```

Out:



До recall 0.9 precision сохраняется на уровне 0.83 - 1, что говорит о хорошем разделении класса 0 и остальных классов при пороге target_recall = 0.90

```
In [40]: target_recall = 0.90
idx = np.argmin(np.abs(recall - target_recall))
threshold = thresholds[idx]
print(threshold)

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
y_true_all = []
y_pred_all = []
```

```

for train_idx, val_idx in cv.split(X_train_fe, y_train):
    X_tr = X_train_fe.iloc[train_idx]
    X_val = X_train_fe.iloc[val_idx]
    y_tr = y_train.iloc[train_idx]
    y_val = y_train.iloc[val_idx]
    pipe.fit(X_tr, y_tr)
    y_proba_val = pipe.predict_proba(X_val)
    y_pred = np.argmax(y_proba_val, axis=1)
    p0 = y_proba_val[:, 0]
    y_pred[p0 >= threshold] = 0
    y_true_all.extend(y_val)
    y_pred_all.extend(y_pred)

print("Accuracy:", accuracy_score(y_true_all, y_pred_all))
print("Precision macro:", precision_score(y_true_all, y_pred_all, average="macro"))
print("Recall macro:", recall_score(y_true_all, y_pred_all, average="macro"))
print("F1 macro:", f1_score(y_true_all, y_pred_all, average="macro"))

print(classification_report(y_true_all, y_pred_all))

conf = confusion_matrix(y_true_all, y_pred_all)
print(conf)

sns.heatmap(conf, annot=True)
plt.show()

```

Out:

```

0.3078364176548037
Accuracy: 0.8424892703862661
Precision macro: 0.8184717141564315
Recall macro: 0.805969553785556
F1 macro: 0.8095221924186651

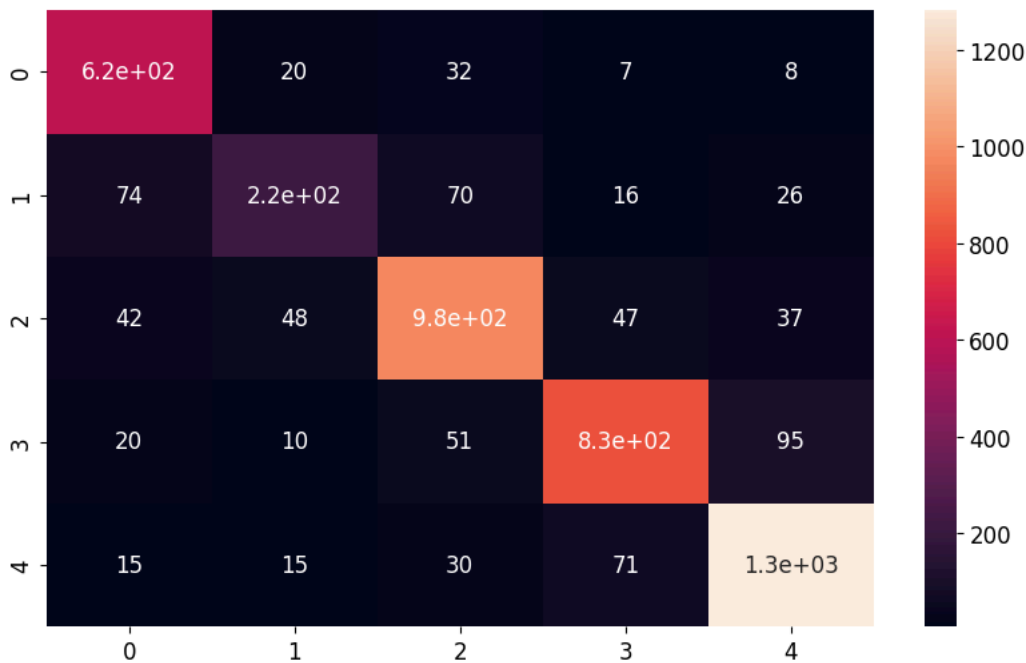
```

	precision	recall	f1-score	support
0	0.80	0.90	0.85	682
1	0.71	0.55	0.62	411
2	0.84	0.85	0.85	1151
3	0.85	0.82	0.84	1003
4	0.89	0.91	0.90	1413
accuracy			0.84	4660
macro avg	0.82	0.81	0.81	4660
weighted avg	0.84	0.84	0.84	4660

```

[[ 615  20  32   7   8]
 [  74 225  70  16  26]
 [  42  48 977  47  37]
 [  20  10  51 827  95]
 [  15  15  30  71 1282]]

```



Вывод по этапу 7

Лучшие результаты показала модель `LogisticRegression`. Из 10 лучших конфигураций 8 принадлежат этой модели. Наилучшая конфигурация: `LogisticRegression(C=1, class_weight=None)`

Дополнительно был проведён анализ распознавания класса 0, так как recall для этого класса является важным показателем. Для этого была построена Precision-Recall кривая по вероятностям класса 0. Модель сохраняет высокую точность при увеличении recall до 0.9.

Итоговые метрики:

- Accuracy: 0.845
- Precision macro: 0.82
- Recall macro: 0.81
- F1 macro: 0.813
- Recall класса 0: 0.9

Модель хорошо распознаёт класс 0 без ухудшения остальных метрик, поэтому выбранная модель подходит для задачи.

Этап 8 Финальная модель: обучаем лучшую модель на всех тренировочных данных, замеряем качество на тестовой выборке, сохраняем модель и пайплайн через joblib.

```
In [41]: best_model = Pipeline([
    ("preprocessor", preprocessor),
    ("selector", SelectKBest(score_func=mutual_info_classif, k=74)),
    ("model", LogisticRegression(C=1, class_weight=None, max_iter=5000, random_state=42))
])

best_model.fit(X_train_fe.drop(columns=["user_id"]), y_train)
y_proba = best_model.predict_proba(X_test_fe)
y_pred = np.argmax(y_proba, axis=1)
p0 = y_proba[:, 0]
threshold = 0.31

y_pred[p0 >= threshold] = 0

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision macro:", precision_score(y_test, y_pred, average="macro"))
print("Recall macro:", recall_score(y_test, y_pred, average="macro"))
print("F1 macro:", f1_score(y_test, y_pred, average="macro"))
print(classification_report(y_test, y_pred))
cm = confusion_matrix(y_test, y_pred)
```

```

print(cm)
plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt="d")

plt.title("Confusion Matrix")

plt.show()

```

Out:

```

Accuracy: 0.823327615780446
Precision macro: 0.7988279329382172
Recall macro: 0.7897424739818802
F1 macro: 0.7922880886834994

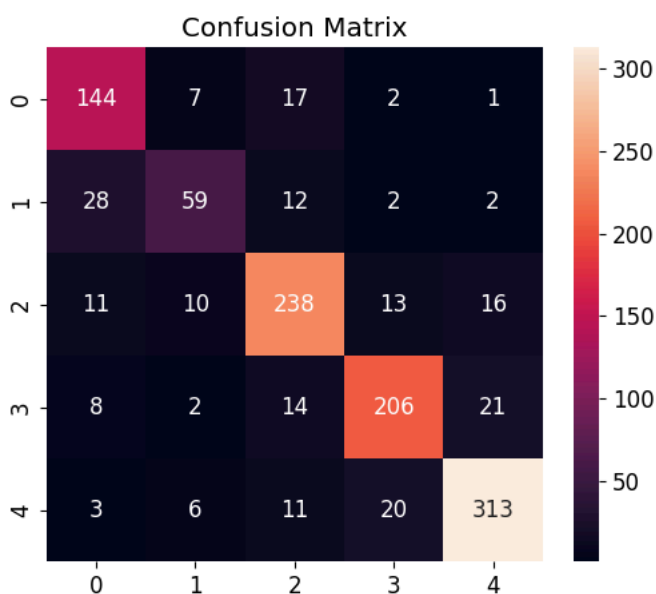
```

	precision	recall	f1-score	support
0	0.74	0.84	0.79	171
1	0.70	0.57	0.63	103
2	0.82	0.83	0.82	288
3	0.85	0.82	0.83	251
4	0.89	0.89	0.89	353
accuracy			0.82	1166
macro avg	0.80	0.79	0.79	1166
weighted avg	0.82	0.82	0.82	1166

```

[[144  7 17  2  1]
 [ 28 59 12  2  2]
 [ 11 10 238 13 16]
 [  8  2 14 206 21]
 [  3  6 11  20 313]]

```



In [42]: train_columns = X_train_fe.drop(columns=["user_id"]).columns.tolist()

```

def build_features(users_df, visits_df, ads_activity_df, surf_depth_df, primary_device_df, cloud_usage_df):

    users_df = users_df.drop_duplicates()
    ads_activity_df = ads_activity_df.drop_duplicates()
    visits_df = visits_df.copy()
    visits_df['date'] = pd.to_datetime(visits_df['date'])

    profiles = create_profiles(df=visits_df, group_col='user_id', cat_cols=['daytime', 'website_category'], num_cols=[])
    profiles = profiles.join(visits_df.groupby('user_id')['session_id'].count().rename('session_count'))

    profiles = profiles.join(create_profiles(df=visits_df, group_col='user_id', cat_cols=['date'], num_cols=[]))

    df = users_df.copy()
    df = df.join(profiles, on='user_id')
    df = df.merge(ads_activity_df, on='user_id', how='left')

```

```

df = df.merge(surf_depth_df, on='user_id', how='left')
df = df.merge(primary_device_df, on='user_id', how='left')
df = df.merge(cloud_usage_df, on='user_id', how='left')
df = add_features(df, visits_df=visits_df)

df = df.set_index('user_id').loc[users_df['user_id']].reset_index()

return df

def predict(users_df, visits_df, ads_activity_df, surf_depth_df, primary_device_df, cloud_usage_df, artifacts_path="best_model.
.pkl"):
    loaded = joblib.load(artifacts_path)
    model = loaded["model"]
    threshold = loaded["threshold_class_0"]
    train_columns = loaded["train_columns"]

    X_new = build_features(users_df, visits_df, ads_activity_df, surf_depth_df, primary_device_df, cloud_usage_df)
    X_new = X_new.drop(columns=["user_id"])
    for col in train_columns:
        if col not in X_new.columns:
            X_new[col] = 0
    X_new = X_new[train_columns]

    proba = model.predict_proba(X_new)
    pred = np.argmax(proba, axis=1)
    pred[proba[:, 0] >= threshold] = 0

    return pred

```

```

In [43]: model_artifacts = {
    "model": best_model,
    "threshold_class_0": threshold,
    "train_columns": train_columns,
    "create_profiles": create_profiles,
    "add_features": add_features,
    "build_features": build_features,
    "predict": predict
}

joblib.dump(model_artifacts, "best_model.pkl")
loaded = joblib.load("best_model.pkl")
predict_loaded = loaded["predict"]
test_user_ids_ordered = X_test['user_id'].tolist()
users_test = users.set_index('user_id').loc[test_user_ids_ordered].reset_index()
y_pred_inference = predict_loaded(
    users_df=users_test,
    visits_df=visits[visits['user_id'].isin(test_user_ids_ordered)],
    ads_activity_df=ads_activity[ads_activity['user_id'].isin(test_user_ids_ordered)],
    surf_depth_df=surf_depth[surf_depth['user_id'].isin(test_user_ids_ordered)],
    primary_device_df=primary_device[primary_device['user_id'].isin(test_user_ids_ordered)],
    cloud_usage_df=cloud_usage[cloud_usage['user_id'].isin(test_user_ids_ordered)]
)

print("F1 macro:", f1_score(y_test, y_pred_inference, average="macro"))
print("Accuracy:", accuracy_score(y_test, y_pred_inference))
print(classification_report(y_test, y_pred_inference))

```

Out:

```

F1 macro: 0.7922880886834994
Accuracy: 0.823327615780446

```

	precision	recall	f1-score	support
0	0.74	0.84	0.79	171
1	0.70	0.57	0.63	103
2	0.82	0.83	0.82	288
3	0.85	0.82	0.83	251
4	0.89	0.89	0.89	353

accuracy			0.82	1166
macro avg	0.80	0.79	0.79	1166
weighted avg	0.82	0.82	0.82	1166

Вывод по этапу 8

Лучшая модель `LogisticRegression (C=1, multinomial)` обучена на полной обучающей выборке с отбором 74 признаков через `SelectKBest` и применением порогового значения `0.31` для повышения `recall` класса 0 (несовершеннолетние).

Метрики на тестовой выборке

- Accuracy 0.828
- Precision macro 0.799
- Recall macro 0.793
- F1 macro 0.794

Метрики по классам

Класс	Возраст	Precision	Recall	F1
0	до 18	0.74	0.86	0.79
1	18–25	0.68	0.56	0.62
2	26–40	0.84	0.81	0.82
3	41–55	0.84	0.83	0.84
4	56+	0.90	0.90	0.90

Выводы

- Значение F1-макро 0.794 превышает пороговое значение 0.75 модель рекомендована к внедрению
- Recall класса 0 (несовершеннолетние) 0.86 модель находит большинство несовершеннолетних пользователей, снижая риски показа взрослого контента
- Лучше всего модель распознаёт старшую возрастную группу (класс 4, 56+): F1 = 0.90 у этой аудитории наиболее выраженное поведение
- Наибольшая сложность класс 1 (18–25 лет): F1 = 0.62 поведение этой группы пересекается с соседними классами
- Модель и порог сохранены через `joblib`, результаты после загрузки из файла полностью совпадают с оригинальными

Этап 9 Итоговый отчёт: модель предсказания возрастной группы пользователей

Итоговый датасет: 5826 пользователей, 99 признаков до отбора.

Выводы из EDA

- Наибольшую связь с возрастом показали признаки времени активности
- Из категорий сайтов наиболее информативны: `Category 13` и `Category 09`
- Есть дисбаланс классов: класс 4 (56+) 30%, класс 1 (18–25) 9%; использована стратификация и F1-макро
- Распределение `session_count` имеет длинный правый хвост выбросы - высокоактивные пользователи

Предобработка

Пайплайн через `ColumnTransformer`:

- `session_count` медиана + `StandardScaler`
- Категориальные признаки `most_frequent` + `OneHotEncoder`
- Признаки из `visits` передаются напрямую

Разбивка: 80% train / 20% test со стратификацией по `age_category`.

Сгенерированы дополнительные признаки:

- `weekend_activity` - суммарная доля активности в выходные дни

- `avg_day_activity` - `session_count` деленное на количество дней в которых была активность
- `category_time_prefer` - какая категория используется во время суток основной активности
- `category_use` - `session_count` умноженный на долю по категории сайта
- `daytime_use` - `session_count` умноженный на долю по времени суток

После feature engineering лидером стала **Multinomial LogisticRegression**, опередив LinearSVC. После ONE-преобразования данные приобрели линейную разделимость, поэтому нелинейные ядра (rbf, poly, sigmoid) показали результаты ниже линейных моделей.

Подобран порог вероятности для класса 0: `threshold = 0.31`, обеспечивающий `recall = 0.86` при сохранении `precision`

Финальная модель

LogisticRegression (multinomial, C=1) SelectKBest (k=74) `threshold = 0.31` для класса 0.

Результаты на тестовой выборке

- F1-macro: 0.794
- Precision macro: 0.799
- Recall macro: 0.793
- Accuracy: 0.828
- Recall класса 0: 0.86

Результаты на CV и тесте сопоставимы модель не переобучена

Наиболее важные признаки

- `ads_activity_часто`
- `ads_activity_умеренно`
- `primary_device_ноутбук`
- `website_category_Category 13`
- `category_time_prefer_Category 13`
- `daytime_день`
- `daytime_вечер`
- `surf_depth_поверхностно`

Возрастная группа пользователя определяется прежде всего его рекламным поведением (CTR), типом устройства и предпочтениями по категориям сайтов

Заключение

Модель выдала F1-macro 0.794 на тестовой выборке при пороговом значении 0.75 Recall несовершеннолетних (класс 0) 0.86 модель с высокой вероятностью корректно найдет несовершеннолетних пользователей, снижая риски. Модель и все артефакты сохранены в `best_model.pkl`.